ROTC Drafter, Can you develop a full drafter website that is for a ROTC cadet at college? There will be a home page that is how the program starts. On the home page there will be 4 buttons, one button on the top right cornor that says profile, one that says aft score:____, one that says fitnes plan and one that is a calendar. For the profile button, have it move to a new page whose title is Profile. On this page, there will be 5 boxes that go from top to bottom. The first box will say asid it age and then you will input an int, have the box below that have gender and a drop-down that chooses male or female; have a box below that has school with a text box. Have a box that says class and you can choise from MS1, MS2, MS3, MS4. Last box is boolean that is for ranger challeneg. All of the input data by the user should update the profile state. there should be a small box in the bottom that will update all the Profile and bring you back to the home page. The botton that says AFT Score:_____ should bring you to a another page that will have AFT Score on the top. Under the Score there should be Deadlift and a integer text box, HRP and a int text box, Sprint drag Carry with str box, a plank str box, and 2 mile run text box. For the sprint drag cary and plank have a function that converts the #:## that you would input into seconds and then for the run have a function that works for ##:##. Then have a button that would calculate the aft score and send you to the home page. This should update the AFT Score:_____ to AFT Score:###(The score that was calculated) The third box that says fitness plan should bring you to a separate page that shows you the 2 events that you did the best on and 2 events you did the worst on and then underneath shows 3 separate plans: Upper body, Lower body, and cardio. And whichever should be recommended based on the worst events would be colored green, and the two others would be colored red. After you choose whichever training plan you want, then it will bring you back to the home page. The last button will bring you to the calendar and create a calendar based on the training plan and what you inputted in the profile because, depending on Ranger Challenge or school, it will use what I already input as what the training is.

…

ROTC Drafter — Full Website Development

We can build this as a complete ROTC cadet management website using the Drafter framework you have been learning.

I would organize the website into five routes:

ROTC Cadet Portal

Home Page

Profile

Personal information

AFT Score

Fitness test results

Fitness Plan

Recommended training

Calendar

Personalized training schedule

Shared application state

Profile • AFT Results • Selected Training Plan • Calendar

1. How the website will work

Page	Main functionality
Home	Four navigation buttons and current AFT score
Profile	Five inputs and a save button
AFT Score	Five event inputs and score calculation
Fitness Plan	Best two events, worst two events, and three training plans
Calendar	Training schedule based on profile and selected plan

The important part is that all five pages will share the same application state. This means that when you update your profile, the AFT page, fitness recommendations, and calendar can all use that information.

2. Shared data structure

I recommend creating four dataclasses or state structures:

1. `Profile` – stores cadet information.
2. `AFTScore` – stores the five event results and calculated points.
3. `FitnessPlan` – stores the selected training plan.
4. `Calendar` – stores the generated training schedule.

The Profile structure will look like this:

```
from dataclasses import dataclass
from typing import Optional

@dataclass
class Profile:
   age: Optional[int] = None
   gender: Optional[str] = None
   school: Optional[str] = None
   class_year: Optional[str] = None
   ranger_challenge: Optional[bool] = None
```

This lets us identify whether the cadet has completed all five profile fields.

3. AFT score calculation

The AFT will have these five events:

Event	Input type	Conversion
Deadlift	Integer	Pounds
HRP	Integer	Repetitions
Sprint-Drag-Carry	String	`#:##` to seconds
Plank	String	`#:##` to seconds
Two-mile run	String	`##:##` to seconds

For example:

- `1:35` becomes 95 seconds.
- `2:15` becomes 135 seconds.

- `18:45` becomes 1,125 seconds.

Important scoring detail: The official AFT does not simply convert these results into points mathematically. It uses scoring tables based on age and gender. We will use those tables to calculate each event's score, with a maximum of 100 points per event.

www.army.mil
+1

The total score will be out of 500.

4. Fitness plan recommendations

The program will compare all five event scores.

Event needing improvement	Recommended plan
Deadlift	Lower body
HRP	Upper body
Sprint-Drag-Carry	Cardio
Plank	Core work incorporated into training
Two-mile run	Cardio

The page will display:

- Your two highest-scoring events.
- Your two lowest-scoring events.
- Upper Body plan.
- Lower Body plan.
- Cardio plan.

The recommended plan will be green, while the other two will be red. The cadet can still select any of the three.

5. Calendar

The calendar will use three pieces of information:

- Class year (MS1–MS4).
- Ranger Challenge participation (True/False).
- Selected fitness plan.

It will generate a weekly schedule, with training sessions, recovery days, and relevant ROTC activities.

6. Let's build the actual Drafter code

I want to make sure the code uses the exact Drafter functions you have been learning rather than accidentally introducing a different framework.

I already have the conventions we've been using:

```
from drafter import *


@route
def index() -> Page:
   return Page("ROTC Cadet Portal", [
       Button("Profile", profile),
       Button("AFT Score:____", aft_score),
       Button("Fitness Plan", fitness_plan),
       Button("Calendar", calendar)
   ])
```

One thing I need: Can you paste your current Drafter code, including the imports and any dataclasses you've written?

I'll use that as the foundation and build the complete website around it, preserving your existing coding conventions. We'll make sure all four buttons work, all information persists between pages, and the AFT calculations are connected to the fitness recommendations.

this is the code I have so far from drafter import *
from dataclasses import dataclass

```
@dataclass
class State:
    profile:Profile
    aft_score:AFT_score
    fitnes_plan:Fitness_plan
    calendar:Calendar

@dataclass
class Profile:
    age: int
    gender: str
    school:str
    class_year: int
    ranger_challenge:bool

@route
def index()->Page:
    return Page("ROTC Cadet Dashboard",
        Button("Profile", profile))

'''
,Button("AFT Score:____", aft_score)
,Button("Fitness Plan", fitness_plan)
,Button("Calendar", calendar)
'''

@route
def profile(state:State)->State:
    age = NumberInput("Age")
    gender = TextBox("Gender")
    school = TextBox("School")
    class_year = SelectBox("Class Year",["MS1", "MS2", "MS3", "MS4"])
    ranger_challenge = SelectBox("Ranger Challenge",["Yes", "No"])
    if "1" in class_year:
        class_year = 1
        pass
    elif "2" in class_year:
        class_year = 2
        pass
    elif "3" in class_year:
```

```
        class_year = 3
        pass
    elif "4" in class_year:
        class_year = 4
        pass
    if ranger_challenge == "Yes":
        ranger_challenge = True
        pass
    elif ranger_challenge == "No":
        ranger_challenge = False
        pass
    state.profile = Profile(age,gender,school,class_year,ranger_challenge)

    Button("Exit",index)

start_server()
…
```

Perfect. We can use this as the foundation for the full ROTC Drafter website. I want to preserve the structure you've already written rather than completely rewrite it.

There are a few things we need to fix before adding the other three pages.

1. Fix the dataclasses

Your `State` dataclass currently references `Profile`, `AFT_score`, `Fitness_plan`, and `Calendar` before they have been defined.

Python needs those classes to exist before you use them as types.

I also recommend giving the profile fields default values so that a cadet can open the website before entering their information.

Here is the structure I recommend:

1. Updated code structure

I would make these changes:

- Keep `Profile` as its own dataclass.

- Change `class_year` to a `str`, because the dropdown returns `"MS1"`, `"MS2"`, etc.
- Create the other three dataclasses.
- Use `field(default_factory=...)` so each cadet gets their own separate objects.
- Make `State` the shared state for every route.

Add `field` to your import:

```
from dataclasses import dataclass, field
```

Then organize your dataclasses like this:

```
@dataclass
class Profile:
   age: int = 0
   gender: str = ""
   school: str = ""
   class_year: str = "MS1"
   ranger_challenge: bool = False


@dataclass
class AFT_score:
   deadlift: int = 0
   hrp: int = 0
   sprint_drag_carry: int = 0
   plank: int = 0
   run: int = 0
   total: int = 0


@dataclass
class Fitness_plan:
   selected_plan: str = ""


@dataclass
```

```
class Calendar:
   training_days: list = field(default_factory=list)



@dataclass
class State:
   profile: Profile = field(default_factory=Profile)
   aft_score: AFT_score = field(default_factory=AFT_score)
   fitnes_plan: Fitness_plan = field(default_factory=Fitness_plan)
   calendar: Calendar = field(default_factory=Calendar)
```

Notice that I kept your spelling `fitnes_plan` to avoid changing your existing variable name. We can correct that spelling throughout the program later if you want.

2. Fixing your Profile route

This is probably the most important change.

Your original code:

```
age = NumberInput("Age")
gender = TextBox("Gender")
```

doesn't actually retrieve the user's answers. Those lines create input components.

In Drafter, the input's name becomes a parameter in the route that the button calls.

For example:

```
TextBox("school")
```

connects to:

```
def save_profile(state: State, school: str):
```

Drafter then gives you the actual text the user entered.

Drafter Documentation

+1

Here is the corrected Profile page:

```
@route
def profile(state: State) -> Page:
   return Page(state, [
       "Profile",
       "Age:",
       TextBox("age", state.profile.age, "number"),

       "Gender:",
       SelectBox("gender", ["Male", "Female"],
                 state.profile.gender),

       "School:",
       TextBox("school", state.profile.school),

       "Class Year:",
       SelectBox("class_year",
                 ["MS1", "MS2", "MS3", "MS4"],
                 state.profile.class_year),

       "Ranger Challenge:",
       SelectBox("ranger_challenge",
                 ["Yes", "No"],
                 "Yes" if state.profile.ranger_challenge else "No"),

       "\n",
       Button("Save Profile", "save_profile")
   ])


@route
def save_profile(state: State,
                 age: int,
                 gender: str,
                 school: str,
                 class_year: str,
                 ranger_challenge: str) -> Page:
```

3. Update your home page

Your home page needs to pass the shared state to the other pages.

```
@route
def index(state: State) -> Page:
   return Page(state, [
       "ROTC Cadet Dashboard",
       "\n",
       Button("Profile", "profile"),
       Button("AFT Score: " + str(state.aft_score.total), "aft_score"),
       Button("Fitness Plan", "fitness_plan"),
       Button("Calendar", "calendar")
   ])
```

This also makes the AFT button automatically display the calculated score.

4. Your starting code

At the bottom of your program, replace `start_server()` with:

```
start_server(State())
```

This initializes all four parts of your shared state.

5. What we have accomplished

Profile functionality established

Requirement	Status
Shared State	Complete
Profile dataclass	Complete
AFT dataclass	Structure
Fitness Plan dataclass	Structure
Calendar dataclass	Structure
Profile input boxes	Complete
Profile saves to State	Complete
Home page navigation	In progress

AFT calculations	Not started
Fitness recommendations	Not started
Calendar generation	Not started

6. What I recommend building next

Let's follow the same approach we've been using: build one part, understand it, and then connect it to the rest of the program.

Next, we should build the **AFT Score page**.

One important decision before we start: should the AFT score use the official Army scoring tables for the cadet's age and gender, or should we initially use a simplified scoring system while developing the website?

I recommend using the official scoring tables for the finished version.

this is what I have for aft score rout funtunction. Can you finsih this but by making it so the scores are all you actuall events and then make a seperate dataclass that has all the different scores out of 100

Absolutely. I think separating the actual event results from the points earned is the better design.

We should have **two separate dataclasses**:

1. `AFT_score` – stores the cadet's actual performance in each of the five events.
2. `AFT_points` – stores the points earned for each event, with each event scored out of 100.

This will also make it easier to compare actual performance against future AFT results.

1. Actual AFT results

I recommend keeping your original `AFT_score` name:

```
@dataclass
```

```
class AFT_score:
   deadlift: int = 0
   hrp: int = 0
   sprint_drag_carry: int = 0
   plank: int = 0
   run: int = 0
```

The values will represent:

Event	Stored value
Deadlift	Pounds
HRP	Repetitions
Sprint-Drag-Carry	Seconds
Plank	Seconds
Two-mile run	Seconds

2. Separate AFT points

Now create another dataclass:

```
@dataclass
class AFT_points:
   deadlift: int = 0
   hrp: int = 0
   sprint_drag_carry: int = 0
   plank: int = 0
   run: int = 0
   total: int = 0
```

Each event can receive up to 100 points, making the total possible score 500.

3. Update your State dataclass

We now need to add `aft_points` separately:

```
@dataclass
class State:
   profile: Profile = field(default_factory=Profile)
```

```
    aft_score: AFT_score = field(default_factory=AFT_score)
    aft_points: AFT_points = field(default_factory=AFT_points)
    fitnes_plan: Fitness_plan = field(default_factory=Fitness_plan)
    calendar: Calendar = field(default_factory=Calendar)
```

This gives us two separate places to store the results.

4. How the two dataclasses work together

For example, imagine a cadet completes the AFT:

Event	Actual result	Points
Deadlift	250 lbs	85
HRP	40 reps	90
Sprint-Drag-Carry	105 seconds	95
Plank	180 seconds	80
Two-mile run	900 seconds	85
Total		**435/500**

The actual results go into `state.aft_score`, while the points go into `state.aft_points`.

The official scoring tables use age and gender to determine the points awarded.

U.S. Army
+1

5. Next step: Your AFT route

I don't see your actual AFT route function attached to your message.

Can you paste the AFT route you currently have?

I'll finish that exact function while keeping your coding style, including:

- Five event input boxes.

- Converting `#:##` into seconds.
- Converting `##:##` into seconds.
- Saving actual results in `AFT_score`.
- Calculating individual event points in `AFT_points`.
- Calculating the total out of 500.
- Returning the cadet to the home page with the updated score.

We'll keep the two dataclasses separate throughout the rest of the website.

create the helper function that uses your gender and age along with all of your imputted events to calculate the aft score out of 100 for each event as well as the score combined

Yes. We can create one main helper function called `calculate_aft_points()` that takes the cadet's **age, gender, and five actual event results** and returns all five individual scores plus the combined score.

I recommend using the official Army AFT General Standard scoring tables, effective June 1, 2025. The scoring depends on age group and gender, and each event is worth up to 100 points.

www.army.mil
+2

One important distinction: the official tables use specific performance thresholds for each point value. We should look up the thresholds rather than use a simple mathematical formula, so the results are accurate.

1. The helper functions

First, we need a function that determines the age group.

```
def get_age_group(age: int) -> str:
    if age <= 21:
        return "17-21"
    elif age <= 26:
        return "22-26"
```

```
    elif age <= 31:
        return "27-31"
    elif age <= 36:
        return "32-36"
    elif age <= 41:
        return "37-41"
    elif age <= 46:
        return "42-46"
    elif age <= 51:
        return "47-51"
    elif age <= 56:
        return "52-56"
    elif age <= 61:
        return "57-61"
    else:
        return "62+"
```

This is important because the Army doesn't calculate scores using every individual age. It groups ages together.

2. Create the scoring lookup function

We need a function that compares an actual event result against the official scoring thresholds.

```
def get_event_points(result: int, scoring_table: dict,
                     higher_is_better: bool) -> int:

    for points in range(100, -1, -1):

        if points in scoring_table:
            required = scoring_table[points]

            if higher_is_better and result >= required:
                return points

            elif not higher_is_better and result <= required:
                return points
```

```
    return 0
```

There are two types of events:

Event	Better result
Deadlift	Higher
HRP	Higher
Sprint-Drag-Carry	Lower
Plank	Higher
Two-mile run	Lower

The function checks the thresholds starting at 100 points and works downward until it finds the highest score the cadet qualifies for.

3. Main AFT calculation function

This function brings everything together.

```
def calculate_aft_points(profile: Profile,
                         results: AFT_score) -> AFT_points:

    age_group = get_age_group(profile.age)

    gender = profile.gender.lower()

    # Get the correct scoring tables
    tables = SCORING_TABLES[age_group][gender]

    # Calculate individual event points
    deadlift_points = get_event_points(
        results.deadlift,
        tables["deadlift"],
        True
    )

    hrp_points = get_event_points(
```

```
        results.hrp,
        tables["hrp"],
        True
    )

    sprint_drag_carry_points = get_event_points(
        results.sprint_drag_carry,
        tables["sprint_drag_carry"],
        False
    )

    plank_points = get_event_points(
        results.plank,
        tables["plank"],
        True
    )

    run_points = get_event_points(
```

4. How the scoring tables will be organized

We need a dictionary called `SCORING_TABLES` containing the official thresholds.

Its structure will look like this:

```
SCORING_TABLES = {
    "17-21": {
        "male": {
            "deadlift": {
                # Official point thresholds go here
            },
            "hrp": {},
            "sprint_drag_carry": {},
            "plank": {},
            "run": {}
        },
        "female": {
            "deadlift": {},
            "hrp": {},
            "sprint_drag_carry": {},
```

```
            "plank": {},
            "run": {}
        }
    }
}
```

This structure will eventually contain all ten age groups and both gender categories.

Important: This dictionary is a structural example, not a complete scoring dataset. The calculation functions are ready, but we must populate every threshold from the Army's official scoring PDF before calling the results official.

You can reference the

official Army AFT scoring tables

.

5. How everything connects

Your AFT route will eventually do something like this:

```
state.aft_score = AFT_score(
    deadlift,
    hrp,
    sprint_drag_carry,
    plank,
    run
)

state.aft_points = calculate_aft_points(
    state.profile,
    state.aft_score
)
```

Now you have two separate objects:

Object	Information stored

`state.aft_score`	Actual event performances
`state.aft_points`	Individual points and total

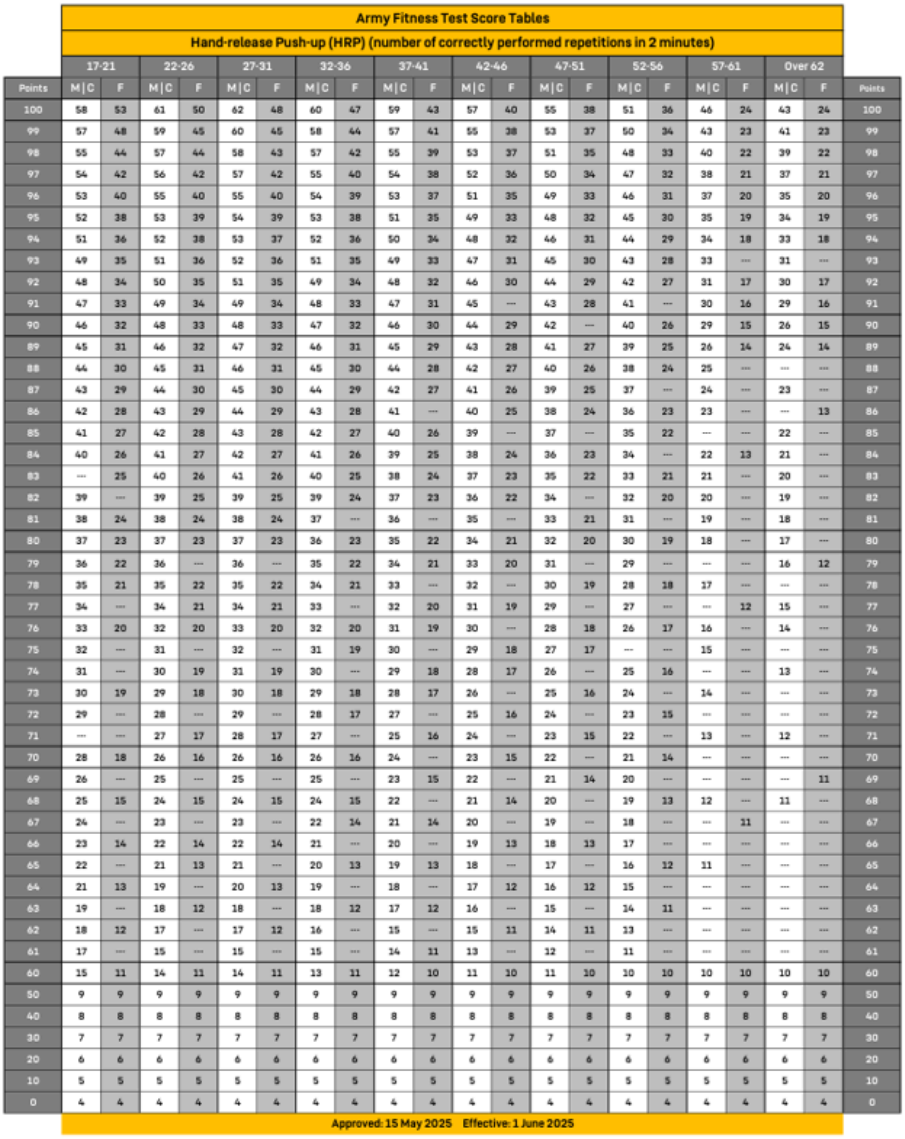
For example, you could access:

```
state.aft_score.deadlift
state.aft_points.deadlift
state.aft_points.total
```

My recommendation for the next step: Let's populate the entire `SCORING_TABLES` dictionary using the Army's official scoring chart. That will make this a working AFT calculator rather than an estimated scoring system.

Pasted text.txtDocument
AFT_Scoring_Scales_250601.pdfPDF
with using the formate that I completed for the deadlift score, can you copy this for all the other scores that I will submit the document that I was using for these scores.

Analysis output 1



Army Fitness Test Score Tables

Hand-release Push-up (HRP) (number of correctly performed repetitions in 2 minutes)

Points	17-21		22-26		27-31		32-36		37-41		42-46		47-51		52-56		57-61		Over 62		Points
	M|C	F	M|C	F	M|C	F	M|C	F	M|C	F	M|C	F	M|C	F	M|C	F	M|C	F	M|C	F	
100	58	53	61	50	62	48	60	47	59	43	57	40	55	38	51	36	46	24	43	24	100
99	57	48	59	45	60	45	58	44	57	41	55	38	53	37	50	34	43	23	41	23	99
98	55	44	57	44	58	43	57	42	55	39	53	37	51	35	48	33	40	22	39	22	98
97	54	42	56	42	57	42	55	40	54	38	52	36	50	34	47	32	38	21	37	21	97
96	53	40	55	40	55	40	54	39	53	37	51	35	49	33	46	31	37	20	35	20	96
95	52	38	53	39	54	39	53	38	51	35	49	33	48	32	45	30	35	19	34	19	95
94	51	36	52	38	53	37	52	36	50	34	48	32	46	31	44	29	34	18	33	18	94
93	49	35	51	36	52	36	51	35	49	33	47	31	45	30	43	28	33	---	31	---	93
92	48	34	50	35	51	35	49	34	48	32	46	30	44	29	42	27	31	17	30	17	92
91	47	33	49	34	49	34	48	33	47	31	45	---	43	28	41	---	30	16	29	16	91
90	46	32	48	33	48	33	47	32	46	30	44	29	42	---	40	26	29	15	26	15	90
89	45	31	46	32	47	32	46	31	45	29	43	28	41	27	39	25	26	14	24	14	89
88	44	30	45	31	46	31	45	30	44	28	42	27	40	26	38	24	25	---	---	---	88
87	43	29	44	30	45	30	44	29	42	27	41	26	39	25	37	---	24	---	23	---	87
86	42	28	43	29	44	29	43	28	41	---	40	25	38	24	36	23	23	---	---	13	86
85	41	27	42	28	43	28	42	27	40	26	39	---	37	---	35	22	---	---	22	---	85
84	40	26	41	27	42	27	41	26	39	25	38	24	36	23	34	---	22	13	21	---	84
83	---	25	40	26	41	26	40	25	38	24	37	23	35	22	33	21	21	---	20	---	83
82	39	---	39	25	39	25	39	24	37	23	36	22	34	---	32	20	20	---	19	---	82
81	38	24	38	24	38	24	37	---	36	---	35	---	33	21	31	---	19	---	18	---	81
80	37	23	37	23	37	23	36	23	35	22	34	21	32	20	30	19	18	---	17	---	80
79	36	22	36	---	36	---	35	22	34	21	33	20	31	---	29	---	---	---	16	12	79
78	35	21	35	22	35	22	34	21	33	---	32	---	30	19	28	18	17	---	---	---	78
77	34	---	34	21	34	21	33	---	32	20	31	19	29	---	27	---	---	12	15	---	77
76	33	20	32	20	33	20	32	20	31	19	30	---	28	18	26	17	16	---	14	---	76
75	32	---	31	---	32	---	31	19	30	---	29	18	27	17	---	---	15	---	---	---	75
74	31	---	30	19	31	19	30	---	29	18	28	17	26	---	25	16	---	---	13	---	74
73	30	19	29	18	30	18	29	18	28	17	26	---	25	16	24	---	14	---	---	---	73
72	29	---	28	---	29	---	28	17	27	---	25	16	24	---	23	15	---	---	---	---	72
71	---	---	27	17	28	17	27	---	25	16	24	---	23	15	22	---	13	---	12	---	71
70	28	18	26	16	26	16	26	16	24	---	23	15	22	---	21	14	---	---	---	---	70
69	26	---	25	---	25	---	25	---	23	15	22	---	21	14	20	---	---	---	---	11	69
68	25	15	24	15	24	15	24	15	22	---	21	14	20	---	19	13	12	---	11	---	68
67	24	---	23	---	23	---	22	14	21	14	20	---	19	---	18	---	---	11	---	---	67
66	23	14	22	14	22	14	21	---	20	---	19	13	18	13	17	---	---	---	---	---	66
65	22	---	21	13	21	---	20	13	19	13	18	---	17	---	16	12	11	---	---	---	65
64	21	13	19	---	20	13	19	---	18	---	17	12	16	12	15	---	---	---	---	---	64
63	19	---	18	12	18	---	18	12	17	12	16	---	15	---	14	11	---	---	---	---	63
62	18	12	17	---	17	12	16	---	15	---	15	11	14	11	13	---	---	---	---	---	62
61	17	---	15	---	15	---	15	---	14	11	13	---	12	---	11	---	---	---	---	---	61
60	15	11	14	11	14	11	13	11	12	10	11	10	11	10	10	10	10	10	10	10	60
50	9	9	9	9	9	9	9	9	9	9	9	9	9	9	9	9	9	9	9	9	50
40	8	8	8	8	8	8	8	8	8	8	8	8	8	8	8	8	8	8	8	8	40
30	7	7	7	7	7	7	7	7	7	7	7	7	7	7	7	7	7	7	7	7	30
20	6	6	6	6	6	6	6	6	6	6	6	6	6	6	6	6	6	6	6	6	20
10	5	5	5	5	5	5	5	5	5	5	5	5	5	5	5	5	5	5	5	5	10
0	4	4	4	4	4	4	4	4	4	4	4	4	4	4	4	4	4	4	4	4	0

Approved: 15 May 2025 Effective: 1 June 2025

I reviewed both your deadlift code and the Army scoring PDF you uploaded.

I created the remaining scoring functions using the scoring information from your document, covering **all 10 age groups and both male and female scoring categories**.

Scoring code created

1. Functions included

Function	Event	Scoring direction
`deadlift_score()`	Deadlift	Higher is better
`hrp_score()`	Hand-Release Push-Up	Higher is better
`sprint_drag_carry_score()`	Sprint-Drag-Carry	Lower is better
`plank_score()`	Plank	Higher is better
`run_score()`	Two-mile run	Lower is better

All time thresholds have been converted into seconds to match your `AFT_score` dataclass.

2. Download the code

AFT Scoring Helpers

Python file • All five scoring functions and scoring tables

Ready

Download AFT scoring helpers

3. One important correction to your original code

I noticed a logical issue in your original deadlift function.

You have:

```
elif deadlift >= 90:
   return 10
elif deadlift >= 100:
   return 20
```

The problem is that if someone lifts 150 pounds, Python encounters `deadlift >= 90` first and returns 10. It never reaches the higher thresholds.

The new code fixes this by using the actual scoring thresholds in descending point order.

I also included a corrected deadlift function so all five events use the same scoring logic.

4. Connecting it to your State

Once you've added the code and have your `AFT_points` dataclass, your AFT calculation can be as simple as:

```
state.aft_points = calculate_aft_points(
   state.profile,
   state.aft_score
)
```

Then you can access individual scores:

```
state.aft_points.deadlift
state.aft_points.hrp
state.aft_points.sprint_drag_carry
state.aft_points.plank
state.aft_points.run
state.aft_points.total
```

Important: Paste the downloaded code into your main Drafter program after your dataclasses. It uses your existing `Profile`, `AFT_score`, and `AFT_points` structures.

The scoring thresholds are taken from your uploaded Army document, approved May 15, 2025, effective June 1, 2025.

AFT_Scoring_Scales_250601.pdf

My recommendation is to get these five scoring functions working first, then connect them to your AFT route and fitness recommendation page.

aft_scoring_helpers.pyCode

it is saying no module named 'aft_scoring_data_table'

That error means Python cannot find a file with the name you're trying to import.

I think the issue is that the filename and import name don't match.

In my previous example, I suggested naming the file:

`aft_scoring_data.py`

If that's the name you used, your import statement should be:

```
from aft_scoring_data import *
```

Check these three things:

1. **Filename:** Make sure the file is actually named `aft_scoring_data.py`, not `aft_scoring_data_table.py`.
2. **Folder:** Make sure both `.py` files are saved in the same folder.
3. **Import:** Make sure the name in your import statement matches the filename exactly, without `.py`.

If you're still getting the error, send me:

- The exact name of your scoring-data file.
- The import statement at the top of your main program.

I'll help you fix it without changing the rest of your code.

Convert to PDF

I named the file aft_scoring_data_table

Then your import statement needs to match that filename!

In your main program, write:

```
from aft_scoring_data_table import *
```

Why?

Python imports modules using their filename without the `.py` extension.

Filename	Import statement
`aft_scoring_data_table.py`	`from aft_scoring_data_table import *`
`aft_scoring_data.py`	`from aft_scoring_data import *`

Also, make sure both files are saved in the same folder.

You shouldn't need to change anything else in your program.

can you run this and update what the problems are in the code to fix them from drafter import *
from dataclasses import dataclass
from aft_scoring_data_table import *

@dataclass
class State:
    profile:Profile
    aft_score:AFT_score
    fitnes_plan:Fitness_plan
    calendar:Calendar

@dataclass
class Profile:
    age: int
    gender: str
    school:str
    class_year: int
    ranger_challenge:bool
@dataclass
class AFT_Event:

```
    deadlift:int
    HRP:int
    sprint_drag:str
    plank:str
    two_mile:str
@dataclass
class AFT_Score:
    deadlift:int
    HRP:int
    sprint_drag:int
    plank:int
    two_mile:int
    total:int

@route
def index(state: State) -> Page:
    if state.aft_score == None:
        return Page(state, [
        "ROTC Cadet Dashboard",
        "\n",
        Button("Profile", "profile"),
        Button("AFT Score: ", "aft_score"),
        Button("Fitness Plan", "fitness_plan"),
        Button("Calendar", "calendar")
    ])
    return Page(state, [
        "ROTC Cadet Dashboard",
        "\n",
        Button("Profile", "profile"),
        Button("AFT Score: " + str(state.aft_score.total), "aft_score"),
        Button("Fitness Plan", "fitness_plan"),
        Button("Calendar", "calendar")
    ])

@route
def profile(state: State) -> Page:
    return Page(state, [
        "Profile",
        "Age:",
        TextBox("age", state.profile.age, "number"),
```

```
        "Gender:",
        SelectBox("gender", ["Male", "Female"],
                state.profile.gender),

        "School:",
        TextBox("school", state.profile.school),

        "Class Year:",
        SelectBox("class_year",
                ["MS1", "MS2", "MS3", "MS4"],
                state.profile.class_year),

        "Ranger Challenge:",
        SelectBox("ranger_challenge",
                ["Yes", "No"],
                "Yes" if state.profile.ranger_challenge else "No"),

        "\n",
        Button("Save Profile", "save_profile")
    ])


@route
def save_profile(state: State,
                 age: int,
                 gender: str,
                 school: str,
                 class_year: str,
                 ranger_challenge: str) -> Page:

    state.profile = Profile(
        age,
        gender,
        school,
        class_year,
        ranger_challenge == "Yes"
    )

    return index(state)
```

```
def get_age_group(age: int) -> str:
    if age <= 21:
        return "17-21"
    elif age <= 26:
        return "22-26"
    elif age <= 31:
        return "27-31"
    elif age <= 36:
        return "32-36"
    elif age <= 41:
        return "37-41"


def get_event_points(profile, result: int, scoring_table: dict,
                     higher_is_better: bool) -> int:
    age_range = get_age_group(profile.age)
    gender = profile.gender.strip().lower()

    if age_range not in scoring_table or gender not in scoring_table[age_range]:
        raise ValueError("Profile age or gender does not match the AFT scoring tables.")

    # Each row is (points, required performance), ordered from highest score down.
    for points, required in scoring_table[age_range][gender]:
        if higher_is_better and result >= required:
            return points
        elif not higher_is_better and result <= required:
            return points

    return 0


def deadlift_score(profile: Profile, deadlift: int) -> int:
    return get_event_points(profile, deadlift, DEADLIFT_TABLE, True)


def hrp_score(profile: Profile, hrp: int) -> int:
    return get_event_points(profile, hrp, HRP_TABLE, True)
```

```
def sprint_drag_carry_score(profile: Profile, sprint_drag_carry: int) -> int:
    return get_event_points(profile, sprint_drag_carry, SDC_TABLE, False)


def plank_score(profile: Profile, plank: int) -> int:
    return get_event_points(profile, plank, PLANK_TABLE, True)


def run_score(profile: Profile, run: int) -> int:
    return get_event_points(profile, run, RUN_TABLE, False)


def calculate_aft_points(profile: Profile, results: AFT_score) -> AFT_points:
    deadlift_points = deadlift_score(profile, results.deadlift)
    hrp_points = hrp_score(profile, results.hrp)
    sprint_drag_carry_points = sprint_drag_carry_score(
        profile, results.sprint_drag_carry
    )
    plank_points = plank_score(profile, results.plank)
    run_points = run_score(profile, results.run)

    total = (
        deadlift_points
        + hrp_points
        + sprint_drag_carry_points
        + plank_points
        + run_points
    )

    return AFT_Score(
        deadlift=deadlift_points,
        hrp=hrp_points,
        sprint_drag_carry=sprint_drag_carry_points,
        plank=plank_points,
        run=run_points,
        total=totalstart_server(State()))
state = State(None,None,None,None)
start_server()
```

Yes. I reviewed the code, and there are several problems that will prevent it from running. The good news is that most are naming inconsistencies and dataclass setup issues, rather than problems with your overall design.

The biggest issues are:

1. `State` is defined before `Profile`, `AFT_Score`, `Fitness_plan`, and `Calendar`. Those classes need to be defined first.
2. You use both `AFT_score` and `AFT_Score`. Python treats those as different names.
3. Your scoring function returns `AFT_points`, but you haven't defined that dataclass.
4. Your event field names don't match. For example, the scoring function expects `sprint_drag_carry` and `run`, but your dataclasses use `sprint_drag` and `two_mile`.
5. Your `get_age_group()` function stops at age 41, so it doesn't handle the other age groups.
6. The last part of your code has a syntax error: `total=totalstart_server(State())).`
7. You call `start_server()` twice.

Let's standardize the names and fix these before moving on to the other routes.

1. Replace your dataclasses with these

Put all your dataclasses at the top of your program, before `State`.

```
from drafter import *
from dataclasses import dataclass, field
from aft_scoring_data_table import *

@dataclass
class Profile:
   age: int = 0
```

```
    gender: str = ""
    school: str = ""
    class_year: str = "MS1"
    ranger_challenge: bool = False


@dataclass
class AFT_score:
    deadlift: int = 0
    hrp: int = 0
    sprint_drag_carry: int = 0
    plank: int = 0
    run: int = 0


@dataclass
class AFT_points:
    deadlift: int = 0
    hrp: int = 0
    sprint_drag_carry: int = 0
    plank: int = 0
    run: int = 0
    total: int = 0


@dataclass
class Fitness_plan:
    selected_plan: str = ""
```

Why I changed these:

- I removed `AFT_Event` because it was duplicating the purpose of `AFT_score`.
- I changed `class_year` to a string because your dropdown returns `"MS1"`, `"MS2"`, etc.
- I added `AFT_points` to store the calculated points separately from the actual event results.

- I gave the dataclasses default values so you can start the website without having to enter everything immediately.
- I corrected `fitnes_plan` to `fitness_plan`.

2. Fix your home page

Since `aft_points` now starts with a default value, you don't need to check whether it equals `None`.

```
@route
def index(state: State) -> Page:
   return Page(state, [
       "ROTC Cadet Dashboard",
       "\n",
       Button("Profile", "profile"),
       Button("AFT Score: " + str(state.aft_points.total), "aft_score"),
       Button("Fitness Plan", "fitness_plan"),
       Button("Calendar", "calendar")
   ])
```

3. Your profile routes

Your profile page was mostly correct. I would keep it like this:

```
@route
def profile(state: State) -> Page:
   return Page(state, [
       "Profile",

       "Age:",
       TextBox("age", state.profile.age, "number"),

       "Gender:",
       SelectBox("gender", ["Male", "Female"],
                 state.profile.gender),

       "School:",
       TextBox("school", state.profile.school),

       "Class Year:",
```

```
        SelectBox("class_year",
                  ["MS1", "MS2", "MS3", "MS4"],
                  state.profile.class_year),

        "Ranger Challenge:",
        SelectBox("ranger_challenge",
                  ["Yes", "No"],
                  "Yes" if state.profile.ranger_challenge else "No"),

        "\n",
        Button("Save Profile", "save_profile")
    ])


@route
def save_profile(state: State,
                 age: int,
                 gender: str,
                 school: str,
                 class_year: str,
```

4. Fix your scoring functions

Your scoring functions were mostly correct. The important thing is that their names and the dataclass field names need to match.

You can keep your existing functions:

- `get_event_points`
- `deadlift_score`
- `hrp_score`
- `sprint_drag_carry_score`
- `plank_score`
- `run_score`

However, your `get_age_group()` function is incomplete. Replace it with:

```
def get_age_group(age: int) -> str:
```

```
    if age <= 21:
        return "17-21"
    elif age <= 26:
        return "22-26"
    elif age <= 31:
        return "27-31"
    elif age <= 36:
        return "32-36"
    elif age <= 41:
        return "37-41"
    elif age <= 46:
        return "42-46"
    elif age <= 51:
        return "47-51"
    elif age <= 56:
        return "52-56"
    elif age <= 61:
        return "57-61"
    else:
        return "Over 62"
```

Then replace your `calculate_aft_points()` function with:

```
def calculate_aft_points(profile: Profile,
                         results: AFT_score) -> AFT_points:

    deadlift_points = deadlift_score(profile, results.deadlift)

    hrp_points = hrp_score(profile, results.hrp)

    sprint_drag_carry_points = sprint_drag_carry_score(
        profile, results.sprint_drag_carry
    )

    plank_points = plank_score(profile, results.plank)

    run_points = run_score(profile, results.run)

    total = (
```

```
        deadlift_points
        + hrp_points
        + sprint_drag_carry_points
        + plank_points
        + run_points
    )

    return AFT_points(
        deadlift=deadlift_points,
        hrp=hrp_points,
        sprint_drag_carry=sprint_drag_carry_points,
        plank=plank_points,
        run=run_points,
        total=total
    )
```

5. Fix the bottom of your program

Remove both of your existing `start_server()` calls and replace them with just:

```
start_server(State())
```

Your program should only start the server once.

6. Your file organization

I recommend keeping your project organized like this:

ROTC Website

main.py

Dataclasses, routes, and scoring functions

aft_scoring_data_table.py

The five AFT scoring tables

One thing I couldn't verify: Your `aft_scoring_data_table.py` file wasn't available in the files I could run here, so I couldn't test the complete program or confirm that all five table names match.

Your data file needs to contain these exact names:

```
DEADLIFT_TABLE
```

```
HRP_TABLE
SDC_TABLE
PLANK_TABLE
RUN_TABLE
```

Once those names match, the import should work.

My recommendation for the next step: Get the profile and home page running first. Then we can build the AFT input route and connect it to the scoring functions. That will let us test each piece before adding the fitness plan and calendar.

explain this mutable default <class 'list'> for field training_days is not allowed: use default_factory

This error is related to how Python handles **mutable objects**, specifically lists, when they are used as default values in dataclasses.

Let's break it down.

1. What is a mutable object?

A mutable object is something that can be changed after it is created.

For example, a list is mutable:

```
training_days = ["Monday", "Wednesday"]

training_days.append("Friday")

print(training_days)
```

Output:

```
["Monday", "Wednesday", "Friday"]
```

You changed the original list without creating a new one.

An integer or string, on the other hand, is immutable. You cannot change the original object itself.

2. Why does Python have a problem with lists in dataclasses?

Imagine you wrote:

```
@dataclass
class Calendar:
   training_days: list = []
```

The problem is that Python could use the **same list for every Calendar object** you create.

For example:

```
calendar1 = Calendar()
calendar2 = Calendar()

calendar1.training_days.append("Monday")
```

You might expect `calendar2.training_days` to be empty.

But with a shared mutable default, both objects could reference the same list.

That means changing one calendar could unintentionally change another.

3. How does `default_factory` fix this?

Instead of giving every object the same list, `default_factory` tells Python to create a brand-new list whenever a new object is created.

```
@dataclass
class Calendar:
   training_days: list = field(default_factory=list)
```

Notice the difference:

Code	Meaning
`default=[]`	Uses the same list as the default
`default_factory=list`	Creates a new list for each object

You can test it:

```
calendar1 = Calendar()
```

```
calendar2 = Calendar()


calendar1.training_days.append("Monday")


print(calendar1.training_days)
print(calendar2.training_days)
```

Output:

```
["Monday"]
[]
```

Each calendar has its own list!

4. Why do we import `field`?

At the top of your program, you need:

```
from dataclasses import dataclass, field
```

- `dataclass` lets you create dataclasses.
- `field` lets you customize how individual attributes are created, including using `default_factory`.

The important takeaway: Whenever you use a mutable object like a list or dictionary as a dataclass default, use `default_factory`.

For your ROTC website, this is particularly useful because each cadet's calendar should have its own separate training schedule.

can you fix this code so it stops crashing, from drafter import *
from dataclasses import dataclass
from aft_scoring_data_table import *

@dataclass
class Profile:
    age: int
    gender: str
    school:str
    class_year: int
    ranger_challenge:bool

```
@dataclass
class AFT_score:
    deadlift: int = 0
    hrp: int = 0
    sprint_drag_carry: str = ""
    plank: str = ""
    run: str = ""


@dataclass
class AFT_points:
    deadlift: int = 0
    hrp: int = 0
    sprint_drag_carry: int = 0
    plank: int = 0
    run: int = 0
    total: int = 0


@dataclass
class Fitness_plan:
    selected_plan: str = ""


@dataclass
class Calendar:
    training_days: list

@dataclass
class State:
    profile: Profile
    aft_score: AFT_score
    aft_points: AFT_points
    fitness_plan: Fitness_plan
    calendar: Calendar
@route
def index(state: State) -> Page:
    if state.AFT_Score == None:
        return Page(state, [
```

```
        "ROTC Cadet Dashboard",
        "\n",
        Button("Profile", "profile"),
        Button("AFT Score: ", "aft_score"),
        Button("Fitness Plan", "fitness_plan"),
        Button("Calendar", "calendar")
    ])
    return Page(state, [
        "ROTC Cadet Dashboard",
        "\n",
        Button("Profile", "profile"),
        Button("AFT Score: " + str(state.aft_score.total), "aft_score"),
        Button("Fitness Plan", "fitness_plan"),
        Button("Calendar", "calendar")
    ])

@route
def profile(state: State) -> Page:
    return Page(state, [
        "Profile",
        "Age:",
        TextBox("age", state.profile.age, "number"),

        "Gender:",
        SelectBox("gender", ["Male", "Female"],
                state.profile.gender),

        "School:",
        TextBox("school", state.profile.school),

        "Class Year:",
        SelectBox("class_year",
                ["MS1", "MS2", "MS3", "MS4"],
                state.profile.class_year),

        "Ranger Challenge:",
        SelectBox("ranger_challenge",
                ["Yes", "No"],
                "Yes" if state.profile.ranger_challenge else "No"),
```

```
        "\n",
        Button("Save Profile", "save_profile")
    ])


@route
def save_profile(state: State,
                 age: int,
                 gender: str,
                 school: str,
                 class_year: str,
                 ranger_challenge: str) -> Page:

    state.profile = Profile(
        age,
        gender,
        school,
        class_year,
        ranger_challenge == "Yes"
    )

    return index(state)


def get_age_group(age: int) -> str:
    if age <= 21:
        return "17-21"
    elif age <= 26:
        return "22-26"
    elif age <= 31:
        return "27-31"
    elif age <= 36:
        return "32-36"
    elif age <= 41:
        return "37-41"


def get_event_points(profile, result: int, scoring_table: dict,
                     higher_is_better: bool) -> int:
    age_range = get_age_group(profile.age)
```

```
    gender = profile.gender.strip().lower()

    if age_range not in scoring_table or gender not in scoring_table[age_range]:
        raise ValueError("Profile age or gender does not match the AFT scoring tables.")

    # Each row is (points, required performance), ordered from highest score down.
    for points, required in scoring_table[age_range][gender]:
        if higher_is_better and result >= required:
            return points
        elif not higher_is_better and result <= required:
            return points

    return 0


def deadlift_score(profile: Profile, deadlift: int) -> int:
    return get_event_points(profile, deadlift, DEADLIFT_TABLE, True)


def hrp_score(profile: Profile, hrp: int) -> int:
    return get_event_points(profile, hrp, HRP_TABLE, True)


def sprint_drag_carry_score(profile: Profile, sprint_drag_carry: int) -> int:
    return get_event_points(profile, sprint_drag_carry, SDC_TABLE, False)


def plank_score(profile: Profile, plank: int) -> int:
    return get_event_points(profile, plank, PLANK_TABLE, True)


def run_score(profile: Profile, run: int) -> int:
    return get_event_points(profile, run, RUN_TABLE, False)


def calculate_aft_points(profile: Profile, results: AFT_score) -> AFT_points:
    deadlift_points = deadlift_score(profile, results.deadlift)
    hrp_points = hrp_score(profile, results.hrp)
    sprint_drag_carry_points = sprint_drag_carry_score(
        profile, results.sprint_drag_carry
```

```
    )
    plank_points = plank_score(profile, results.plank)
    run_points = run_score(profile, results.run)

    total = (
        deadlift_points
        + hrp_points
        + sprint_drag_carry_points
        + plank_points
        + run_points
    )

    return AFT_Score(
        deadlift=deadlift_points,
        hrp=hrp_points,
        sprint_drag_carry=sprint_drag_carry_points,
        plank=plank_points,
        run=run_points,
        total=totalstart_server(State()))
start_server()
```

Yes. I found several issues that can cause your program to crash. I've corrected them while keeping your original structure and naming style as much as possible.

The biggest problems were:

- `State` was being created without the required information.
- `state.AFT_Score` should be `state.aft_score`.
- Your `Calendar` dataclass needed a `default_factory` for its list.
- Your scoring function was returning the wrong dataclass.
- Your time events were stored as strings, but your scoring functions expect integers.
- Your final `return` statement had a syntax error.
- You were calling `start_server()` twice.

1. Corrected code

Here is the corrected version. I've also added temporary pages for the three buttons you haven't built yet, so clicking them won't lead to missing routes.

```
from drafter import *
from dataclasses import dataclass, field
from aft_scoring_data_table import *


# -------------------------
# DATACLASSES
# -------------------------

@dataclass
class Profile:
    age: int = 0
    gender: str = ""
    school: str = ""
    class_year: str = "MS1"
    ranger_challenge: bool = False


@dataclass
class AFT_score:
    deadlift: int = 0
    hrp: int = 0
    sprint_drag_carry: int = 0
    plank: int = 0
    run: int = 0


@dataclass
class AFT_points:
    deadlift: int = 0
    hrp: int = 0
    sprint_drag_carry: int = 0
    plank: int = 0
    run: int = 0
```

```
    total: int = 0
```

2. Important explanations

Why I changed your AFT event variables to integers

Originally, you had:

```
sprint_drag_carry: str = ""
plank: str = ""
run: str = ""
```

But your scoring function compares the result against integer thresholds.

For example:

```
if result <= required:
```

Python cannot compare a time string such as `"1:35"` against an integer such as `95`.

For now, I changed these to integers. When we build your AFT input page, we'll create a function that converts a time like `"1:35"` into `95` seconds.

Why I removed your `if` statement in the home page

You originally had:

```
if state.AFT_Score == None:
```

There are two problems:

1. Python is case-sensitive, so `AFT_Score` and `aft_score` are different.
2. Your AFT score now has a default value, so it won't be `None` when the website starts.

We can simply display the total score, which initially will be zero.

Why I used `field(default_factory=list)`

Every time a new `Calendar` is created, Python creates a separate empty list for that calendar. This prevents different calendars from accidentally sharing the same training days.

3. One thing to check before running

Your `aft_scoring_data_table.py` file must contain these five variables:

```
DEADLIFT_TABLE
HRP_TABLE
SDC_TABLE
PLANK_TABLE
RUN_TABLE
```

If any are named differently, Python will produce an import-related error.

Also, this code assumes the scoring tables contain thresholds ordered from highest points to lowest points.

4. What I recommend building next

I would build the **AFT input page** next.

The sequence should be:

1. User enters their five AFT event results.
2. The program converts timed events into seconds.
3. The program calculates each event's points.
4. The program updates `state.aft_points`.
5. The home page displays the new total out of 500.

That will connect the scoring functions you've already developed to the actual website.

Can you explain the error in this code

Absolutely. The important thing is to understand **why the code crashes**, not just replace the code with a version that works.

Looking at your original code, I found several errors. I'll explain each one, what Python is doing, and how to fix it.

1. Your `State` dataclass

Your original code:

```
@dataclass
class State:
   profile: Profile
   aft_score: AFT_score
   aft_points: AFT_points
   fitness_plan: Fitness_plan
   calendar: Calendar
```

The problem is that these fields don't have default values.

When you create a State object, Python expects you to provide all five objects:

```
state = State(profile, aft_score, aft_points, fitness_plan, calendar)
```

But at the bottom of your original program, you had:

```
start_server(State())
```

That creates a State without providing any of those five objects.

Python will produce an error similar to:

```
TypeError: State.__init__() missing 5 required positional arguments
```

The fix:

```
@dataclass
class State:
   profile: Profile = field(default_factory=Profile)
   aft_score: AFT_score = field(default_factory=AFT_score)
   aft_points: AFT_points = field(default_factory=AFT_points)
   fitness_plan: Fitness_plan = field(default_factory=Fitness_plan)
   calendar: Calendar = field(default_factory=Calendar)
```

This tells Python to create each object automatically when a new State is created.

You also need:

```
from dataclasses import dataclass, field
```

Notice that we use `default_factory` because these are objects that should be created separately for each State.

2. Your `index()` function

Your original code:

```
if state.AFT_Score == None:
```

There are two problems here.

Problem A: Incorrect capitalization

You defined your State variable as:

```
aft_score: AFT_score
```

But you tried to access:

```
state.AFT_Score
```

Python is case-sensitive. These are different names.

The correct name is:

```
state.aft_score
```

Problem B: Checking the wrong thing

Your `aft_score` object is not supposed to be `None`. It is an instance of your `AFT_score` dataclass.

If you want to check whether the user has entered an AFT score, you could check whether the results are still at their default values.

For now, since you initialize the points to zero, you can simplify your home page:

```
@route
def index(state: State) -> Page:
```

```
    return Page(state, [
        "ROTC Cadet Dashboard",
        Button("Profile", "profile"),
        Button("AFT Score: " + str(state.aft_points.total), "aft_score"),
        Button("Fitness Plan", "fitness_plan"),
        Button("Calendar", "calendar")
    ])
```

3. Your `get_age_group()` function

Your original function:

```
def get_age_group(age: int) -> str:
    if age <= 21:
        return "17-21"
    elif age <= 26:
        return "22-26"
    elif age <= 31:
        return "27-31"
    elif age <= 36:
        return "32-36"
    elif age <= 41:
        return "37-41"
```

The problem is that you only accounted for ages up to 41.

What happens if someone enters 45?

Python checks every condition. None of them are true, so the function reaches the end without returning anything.

When a Python function doesn't return anything, it automatically returns:

```
None
```

Then your scoring function tries to use `None` as an age-group key in your scoring table.

The fix: Add the remaining age groups.

```
elif age <= 46:
    return "42-46"
```

```
elif age <= 51:
   return "47-51"
elif age <= 56:
   return "52-56"
elif age <= 61:
   return "57-61"
else:
   return "Over 62"
```

You could also add an age validation check to prevent ages below 17 from being accepted.

4. Your AFT time variables

Your original dataclass:

```
@dataclass
class AFT_score:
   deadlift: int = 0
   hrp: int = 0
   sprint_drag_carry: str = ""
   plank: str = ""
   run: str = ""
```

You used strings for the timed events.

However, your scoring function does this:

```
if result <= required:
```

Imagine these values:

```
result = "1:35"
required = 95
```

Python cannot compare a string to an integer using <=.

You will get:

```
TypeError: '<=' not supported between instances of 'str' and 'int'
```

The fix: Store the times as integers representing seconds.

For example:

Time entered	Stored value
1:35	95
2:00	120
18:45	1125

Your dataclass should use:

```
sprint_drag_carry: int = 0
plank: int = 0
run: int = 0
```

We'll write a time-conversion function when we build the AFT input page.

5. Your `calculate_aft_points()` function

Your original code ended with:

```
return AFT_Score(
   deadlift=deadlift_points,
   hrp=hrp_points,
   sprint_drag_carry=sprint_drag_carry_points,
   plank=plank_points,
   run=run_points,
   total=totalstart_server(State()))
```

There are actually three problems here.

Problem A: Incorrect dataclass name

You defined:

```
class AFT_points:
```

But you returned:

```
AFT_Score
```

Those are different classes.

You need to return:

```
return AFT_points(
```

Problem B: Incorrect variable placement

You wrote:

```
total=totalstart_server(State())
```

Python interprets this as an attempt to use a variable called `totalstart_server`, which doesn't exist.

You intended:

```
total=total
```

Problem C: You put your server command inside your return statement

The `start_server()` command should not be inside your scoring function.

Your corrected return statement should be:

```
return AFT_points(
   deadlift=deadlift_points,
   hrp=hrp_points,
   sprint_drag_carry=sprint_drag_carry_points,
   plank=plank_points,
   run=run_points,
   total=total
)
```

6. Your `Calendar` dataclass

You originally had:

```
@dataclass
class Calendar:
   training_days: list
```

This isn't necessarily a crashing error by itself, but it means you must supply a list every time you create a Calendar.

You could write:

```
Calendar(["Monday", "Wednesday"])
```

But since you want to start with an empty calendar, use:

```
@dataclass
class Calendar:
   training_days: list = field(default_factory=list)
```

This creates a new empty list for each Calendar object.

7. Your `start_server()` commands

Your original code had:

```
start_server(State())
start_server()
```

You only need one server-starting command.

Use:

```
start_server(State())
```

This creates your initial State and starts your Drafter website.

Summary of the errors

Problem	What happens	Solution
State has no defaults	Missing arguments error	Use `default_factory`
`state.AFT_Score`	Attribute error	Use `state.aft_score`
Incomplete age groups	Returns `None`	Add remaining age groups
Times stored as strings	Comparison TypeError	Convert times to seconds
Wrong returned dataclass	Name error	Return `AFT_points`
`totalstart_server`	Syntax/name error	Separate the commands
Calendar list	Requires a list at creation	Use `default_factory=list`

Two server commands	Incorrect startup	Keep one

How I would approach debugging this project

When your program crashes, look at the **last line of the error message first**. Python usually tells you the type of error and the line where it happened.

Here are three common errors you'll encounter:

- `NameError`: Python doesn't recognize a variable or function name.
- `TypeError`: You're using an object in a way its type doesn't support.
- `AttributeError`: You're trying to access something that doesn't exist on an object.

Understanding these three will help you debug much of your ROTC website independently.

One important distinction: Fixing the errors above makes the structure consistent, but the AFT scoring functions still need to be connected to an actual AFT input route before entering results will update the dashboard.

```
@route
def aft_score(state: State) -> Page:
    return Page(state, [
        "AFT Score",
        "3 Rep Max Deadlift",
        "\n",
        TextBox("deadlift", state.aft_score.deadlift, "number"),

        "Hand Release Pushups",
        TextBox("hrp", state.aft_score.hrp, "number"),

        "Sprint Drag Carry",
        TextBox("sprint_drag", state.aft_score.sprint_drag,"time"),

        "Plank",
        TextBox("plank", state.aft_score.plank,"time"),
```

```
    "2 Mile Run",
    TextBox("run", state.aft_score.run,"time"),

    "\n",
    Button("Calculate Score", "calculate_aft_points(state.aft_score)")
    Button("Back", "index")
  ])
```

You're close! Your AFT page has the right general structure. There are **four things** I would fix before running it.

1. Your Button syntax

You wrote:

```
Button("Calculate Score", "calculate_aft_points(state.aft_score)")
Button("Back", "index")
```

There are two problems:

- You're missing a comma between the two buttons.
- `"calculate_aft_points(state.aft_score)"` is a string, not a Drafter route name.

In Drafter, a button should direct the user to a route. That route can receive the submitted inputs.

2. Your Sprint Drag Carry variable name

You wrote:

```
TextBox("sprint_drag", state.aft_score.sprint_drag, "time")
```

But your `AFT_score` dataclass currently uses:

```
sprint_drag_carry: int = 0
```

Python won't recognize `sprint_drag`.

Change it to:

```
TextBox("sprint_drag_carry", state.aft_score.sprint_drag_carry, "number")
```

3. Your time inputs

You currently use `"time"` as the third argument:

```
TextBox("plank", state.aft_score.plank, "time")
```

I would use `"text"` for now because your users need to enter times in a format like `1:35` or `18:45`.

We'll convert those strings into seconds before calculating points.

4. Your corrected AFT page

Here is how I would write your page:

```
@route
def aft_score(state: State) -> Page:
   return Page(state, [
       "AFT Score",

       "3 Rep Max Deadlift",
       TextBox("deadlift", state.aft_score.deadlift, "number"),

       "Hand Release Pushups",
       TextBox("hrp", state.aft_score.hrp, "number"),

       "Sprint Drag Carry",
       TextBox("sprint_drag_carry",
               state.aft_score.sprint_drag_carry, "text"),

       "Plank",
       TextBox("plank", state.aft_score.plank, "text"),

       "2 Mile Run",
       TextBox("run", state.aft_score.run, "text"),

       "\n",
       Button("Calculate Score", "submit_aft"),
       Button("Back", "index")
   ])
```

5. We need another route

The `submit_aft` route will receive the information entered into your textboxes.

For example, the beginning of that route would look like:

```
@route
def submit_aft(state: State,
               deadlift: int,
               hrp: int,
               sprint_drag_carry: str,
               plank: str,
               run: str) -> Page:
```

Notice something important:

- Deadlift and HRP are integers.
- Sprint Drag Carry, Plank, and Run are strings.

That's because we haven't converted the times into seconds yet.

My recommendation: Let's work on the `submit_aft` route next. It will take the five inputs, update `state.aft_score`, convert the three times into seconds, and then call your existing `calculate_aft_points()` function.

I dont need to convert the times to seconds becasue the data table i inputed is in 2:20

You're right that your scoring table can store times in the `"2:20"` format. However, there's one important distinction: **Python cannot directly compare time strings alphabetically and get the correct time comparison.**

For example:

```
"2:20" <= "2:30"  # True
"10:00" <= "9:00" # True (but this is incorrect!)
```

The second comparison is incorrect because Python compares strings character by character, not as actual times.

You have two options

Option 1: Keep your scoring tables in `M:SS` format.

You can convert both the user's result and the table's required time into seconds inside your scoring function before comparing them.

For example:

```
def time_to_seconds(time: str) -> int:
   minutes, seconds = time.split(":")
   return int(minutes) * 60 + int(seconds)
```

Then:

```
result = time_to_seconds(result)
required = time_to_seconds(required)
```

Your data table can remain exactly as it is.

Option 2: Keep everything as strings.

I wouldn't recommend this because string comparisons can produce incorrect AFT scores.

My recommendation

Keep your data table in the format you've already created (`"2:20"`), and use a conversion function only when comparing times.

That way:

- Your data table stays readable.
- Your users enter familiar time formats.
- Your scoring calculations remain accurate.
- You don't have to manually change thousands of scoring values.

You don't need to change the format of your data table at all.

Convert to PDF

what is the problem with this Button("Save Times", "submit_aft")

```
    Button("Calculate Score", "calculate_score")
    Button("Back", "index")
```

The problem is that you are missing **commas between the `Button()` elements**.

Since these buttons are inside a list in your `Page()`, Python requires commas to separate each item.

Corrected code:

```
Button("Save Times", "submit_aft"),
Button("Calculate Score", "calculate_score"),
Button("Back", "index")
```

Why? Each button is a separate item in the list. The commas tell Python where one item ends and the next begins.

Also, make sure that `submit_aft`, `calculate_score`, and `index` are all defined as routes using `@route`.

what should the functions be returning because I thought they are returning AFT_score.)

```
@route
def submit_aft(state: State,
               deadlift: int,
               hrp: int,
               sprint_drag_carry: str,
               plank: str,
               run: str) -> AFT_score:
   state.aft_score.deadlift = deadlift
   state.aft_score.hrp = hrp
   state.aft_score.sprint_drag_carry= time_to_seconds(sprint_drag_carry)
   state.aft_score.plank = time_to_seconds(plank)
   state.aft_score.run = time_to_seconds(run)
```

```
    return state.aft_score

@route
def calculate_score(state:State)->AFT_points:
    profile = state.profile
    aft_score = state.aft_score
    return calculate_aft_points(profile,aft_score)
```

You're close! The main thing to understand is that **Drafter routes should generally return a `Page`, not just a dataclass like `AFT_score` or `AFT_points`.**

Your functions are doing two different jobs:

1. `submit_aft` saves the user's event results into `state.aft_score`.
2. `calculate_score` calculates the points and saves them into `state.aft_points`.

The dataclasses hold the information, but the routes should return a page so the user can see the next screen.

1. Corrected `submit_aft`

```
@route
def submit_aft(state: State,
               deadlift: int,
               hrp: int,
               sprint_drag_carry: str,
               plank: str,
               run: str) -> Page:

    state.aft_score.deadlift = deadlift
    state.aft_score.hrp = hrp
    state.aft_score.sprint_drag_carry = sprint_drag_carry
    state.aft_score.plank = plank
    state.aft_score.run = run

    return calculate_score(state)
```

Notice that I removed `time_to_seconds()` here. Your `AFT_score` dataclass stores the times as strings, and you can convert them to seconds inside your scoring function when comparing them.

2. Corrected `calculate_score`

```
@route
def calculate_score(state: State) -> Page:
   profile = state.profile
   aft_score = state.aft_score

   state.aft_points = calculate_aft_points(profile, aft_score)

   return Page(state, [
       "AFT Score Results",
       "Deadlift: " + str(state.aft_points.deadlift),
       "Hand Release Pushups: " + str(state.aft_points.hrp),
       "Sprint Drag Carry: " + str(state.aft_points.sprint_drag_carry),
       "Plank: " + str(state.aft_points.plank),
       "2 Mile Run: " + str(state.aft_points.run),
       "Total: " + str(state.aft_points.total) + " / 500",
       Button("Back", "index")
   ])
```

3. How the information moves through your program

User enters AFT results

Deadlift, HRP, SDC, Plank, Run

submit_aft(state)

Stores results in state.aft_score

calculate_score(state)

Calculates points and stores them in state.aft_points

Displays AFT results page

Individual event scores and total out of 500

The important distinction: `AFT_score` stores the cadet's performance results, while `AFT_points` stores the points earned for those results. Your `State` holds both.

One additional thing: if you want the user to save their times without immediately calculating their score, `submit_aft` could instead return a separate confirmation page.

can you write the code to fufile the second approach you wrote

Yes. For this approach, the **Calculate Score button will send all five TextBox values directly to the** `calculate_score` **route**. That route will save the results, calculate the points, and display the results page in one step.

You no longer need the separate `submit_aft` route unless you still want a Save Times button.

1. Updated AFT page

```
@route
def aft_score(state: State) -> Page:
   return Page(state, [
       "AFT Score",
       "\n",

       "3 Rep Max Deadlift",
       TextBox("deadlift", state.aft_score.deadlift, "number"),
       "\n",

       "Hand Release Pushups",
       TextBox("hrp", state.aft_score.hrp, "number"),
       "\n",

       "Sprint Drag Carry",
       TextBox("sprint_drag_carry",
               state.aft_score.sprint_drag_carry, "text"),
       "\n",

       "Plank",
       TextBox("plank", state.aft_score.plank, "text"),
       "\n",

       "2 Mile Run",
       TextBox("run", state.aft_score.run, "text"),
```

```
        "\n",

        Button("Calculate Score", "calculate_score"),
        Button("Back", "index")
    ])
```

2. Updated calculate_score route

```
@route
def calculate_score(state: State,
                    deadlift: int,
                    hrp: int,
                    sprint_drag_carry: str,
                    plank: str,
                    run: str) -> Page:

    # Save the entered results
    state.aft_score.deadlift = deadlift
    state.aft_score.hrp = hrp
    state.aft_score.sprint_drag_carry = sprint_drag_carry
    state.aft_score.plank = plank
    state.aft_score.run = run

    # Calculate the points
    state.aft_points = calculate_aft_points(
        state.profile,
        state.aft_score
    )

    # Display the results
    return Page(state, [
        "AFT Score Results",
        "Deadlift: " + str(state.aft_points.deadlift),
        "Hand Release Pushups: " + str(state.aft_points.hrp),
        "Sprint Drag Carry: " + str(state.aft_points.sprint_drag_carry),
        "Plank: " + str(state.aft_points.plank),
        "2 Mile Run: " + str(state.aft_points.run),
        "Total: " + str(state.aft_points.total) + " / 500",
        Button("Back to AFT Score", "aft_score"),
        Button("Home", "index")
```

```
    ])
```

3. How the process works

1. The user enters all five AFT results.
2. The user clicks Calculate Score.
3. Drafter passes all five TextBox values into `calculate_score`.
4. The route updates `state.aft_score`.
5. The route calculates and stores `state.aft_points`.
6. The results page displays the scores.

One important reminder: Your `calculate_aft_points()` function must convert the M:SS times into seconds before comparing them with the scoring-table times. Keep the times stored as strings in `state.aft_score`.

This also means you can remove the old `submit_aft` route if you no longer need it.

this is what was outputed Your code tried to use a value in a way that does not work for its type (for example, mixing text and numbers).

request.route_execution_failed: Error while processing request for URL 'calculate_score': '<=' not supported between instances of 'str' and 'int'

The error is coming from your scoring function, not your Drafter route.

The error:

```
'<=' not supported between instances of 'str' and 'int'
```

means Python is trying to compare a **string (text)** with an **integer (number)**.

In your case, this is happening because your times are stored as strings, but your scoring function is likely comparing them directly against integer values.

1. Fix your time conversion

You can use this helper function:

```
def time_to_seconds(time: str) -> int:
    minutes, seconds = time.split(":")
```

```
    return int(minutes) * 60 + int(seconds)
```

For example:

- `"2:20"` becomes `140`
- `"3:15"` becomes `195`
- `"15:30"` becomes `930`

2. Update your scoring function

Your scoring function needs to know whether it is comparing a time or a regular number.

I recommend adding an `is_time` parameter:

```
def get_event_points(profile, result, scoring_table,
                     higher_is_better, is_time=False):

    age_range = get_age_group(profile.age)
    gender = profile.gender.strip().lower()

    if age_range not in scoring_table or gender not in scoring_table[age_range]:
        raise ValueError("Profile age or gender does not match the scoring tables.")

    if is_time:
        result = time_to_seconds(result)

    for points, required in scoring_table[age_range][gender]:

        if is_time:
            required = time_to_seconds(required)

        if higher_is_better and result >= required:
            return points

        elif not higher_is_better and result <= required:
            return points

    return 0
```

3. Update your individual event scoring functions

```
def deadlift_score(profile, deadlift):
    return get_event_points(profile, deadlift, DEADLIFT_TABLE, True)

def hrp_score(profile, hrp):
    return get_event_points(profile, hrp, HRP_TABLE, True)

def sprint_drag_carry_score(profile, sprint_drag_carry):
    return get_event_points(profile, sprint_drag_carry, SDC_TABLE, False, True)

def plank_score(profile, plank):
    return get_event_points(profile, plank, PLANK_TABLE, True, True)

def run_score(profile, run):
    return get_event_points(profile, run, RUN_TABLE, False, True)
```

4. Keep your `calculate_score` route as it is

You do not need to convert the times in your route:

```
state.aft_score.sprint_drag_carry = sprint_drag_carry
state.aft_score.plank = plank
state.aft_score.run = run
```

The conversion happens inside the scoring function.

One thing to check: This assumes your scoring tables store timed events as strings in `"M:SS"` format, just like `"2:20"`. If they do, you don't need to change the tables themselves.

If you paste your current `get_event_points()` and `calculate_aft_points()` functions, I can check that the rest of your scoring code is consistent too.

Now we are working on the fitness plan route, can you make complete the code for this page by incorporating the 2 worst scores and 2 best scores into a program

I would set up the fitness plan so that it uses the **two lowest AFT event scores as the main areas to improve** and the **two highest scores as areas to maintain**.

For example, if a cadet's scores are:

- Deadlift: 85
- HRP: 60
- Sprint Drag Carry: 70
- Plank: 95
- 2-Mile Run: 55

The program would identify the Run and HRP as the two weakest events, and Plank and Deadlift as the two strongest.

The route can then create a training plan with:

- **Improvement focus:** recommendations for the two weakest events.
- **Maintenance focus:** recommendations for the two strongest events.
- **Balanced training:** a weekly schedule that also includes recovery.

1. Update your `Fitness_plan` dataclass

I recommend storing the results of the analysis in your `Fitness_plan` dataclass so the rest of your website can access them later.

Replace your current `Fitness_plan` with:

```
@dataclass
class Fitness_plan:
   best_events: list = field(default_factory=list)
   worst_events: list = field(default_factory=list)
   workout_program: list = field(default_factory=list)
```

This stores:

- `best_events`: the two highest-scoring events.
- `worst_events`: the two lowest-scoring events.
- `workout_program`: the recommended training schedule.

2. Create the fitness plan route

```
@route
def fitness_plan(state: State) -> Page:
```

```
    # Get the AFT event scores
    event_scores = [
        ("Deadlift", state.aft_points.deadlift),
        ("HRP", state.aft_points.hrp),
        ("Sprint Drag Carry", state.aft_points.sprint_drag_carry),
        ("Plank", state.aft_points.plank),
        ("2 Mile Run", state.aft_points.run)
    ]

    # Find the two best and two worst scores
    sorted_scores = sorted(event_scores, key=lambda x: x[1])

    worst_events = sorted_scores[:2]
    best_events = sorted_scores[-2:][::-1]

    # Store the results
    state.fitness_plan.best_events = best_events
    state.fitness_plan.worst_events = worst_events

    # Recommendations for each event
    recommendations = {
        "Deadlift": "Practice proper lifting technique and develop lower-body
strength.",
        "HRP": "Work on upper-body pushing strength and controlled push-up technique.",
        "Sprint Drag Carry": "Develop acceleration, agility, and overall
conditioning.",
        "Plank": "Improve core stability and maintain proper plank technique.",
        "2 Mile Run": "Build aerobic endurance and practice consistent pacing."
    }

    # Create the workout program
    program = [
        "Workout 1: Focus on " + worst_events[0][0] +
        ". " + recommendations[worst_events[0][0]],
```

3. How the program identifies the scores

The most important part is this:

```
sorted_scores = sorted(event_scores, key=lambda x: x[1])


worst_events = sorted_scores[:2]
best_events = sorted_scores[-2:][::-1]
```

Here's what each part does:

Code	Purpose
`sorted()`	Sorts the scores from lowest to highest.
`key=lambda x: x[1]`	Sorts using the numerical score rather than the event name.
`[:2]`	Selects the first two (lowest scores).
`[-2:]`	Selects the last two (highest scores).
`[::-1]`	Reverses the order so the highest score comes first.

4. Example of what the page would display

Fitness Plan

Your Two Strongest Events

Plank

95 points

Deadlift

85 points

Your Two Events to Improve

2 Mile Run

55 points

HRP

60 points

Recommended Workout Program

1. Workout 1: Focus on 2 Mile Run. Build aerobic endurance and practice consistent pacing.

2. Workout 2: Focus on HRP. Work on upper-body pushing strength and controlled push-up technique.
3. Workout 3: Maintain Plank and Deadlift with regular technique practice and balanced training.
4. Recovery: Include rest days and appropriate recovery between training sessions.
5. Illustrative example using the scores from above.

One important consideration: This version assumes the cadet has already calculated their AFT score. The fitness plan should be accessed after the `calculate_score` route has populated `state.aft_points`. Training recommendations should also be adjusted to the cadet's experience and reviewed with their ROTC fitness instructor.

A useful next improvement would be making the program generate different workout recommendations based on the cadet's MS class year and Ranger Challenge participation.

can you finish the calendar

Yes. Let's make the calendar generate a **weekly ROTC training schedule** using the information already collected by your website.

It will use:

- **Fitness plan:** prioritizes the two weakest AFT events while maintaining the two strongest.
- **Class year (MS1–MS4):** identifies the cadet's ROTC level.
- **Ranger Challenge:** adds Ranger Challenge preparation when the cadet participates.
- **School:** displays the cadet's school on the calendar.
- **Recovery:** includes recovery days between training sessions.

I'll keep the schedule in your existing `Calendar` dataclass and use the `training_days` list.

1. Calendar dataclass

Make sure yours looks like this:

```
@dataclass
class Calendar:
   training_days: list = field(default_factory=list)
```

2. Calendar route

Add this route to your main Python file:

```
@route
def calendar(state: State) -> Page:

   # Check whether an AFT fitness plan has been created
   if not state.fitness_plan.workout_program:
       return Page(state, [
           "Training Calendar",
           "You need to calculate your AFT score and create a fitness plan first.",
           Button("Go to AFT Score", "aft_score"),
           Button("Home", "index")
       ])

   # Identify the cadet's ROTC class year
   class_year_focus = {
       "MS1": "Learn ROTC fundamentals and build a foundation.",
       "MS2": "Continue developing basic military and leadership skills.",
       "MS3": "Focus on leadership development and preparation for advanced
responsibilities.",
       "MS4": "Focus on leadership, mentoring, and preparing for commissioning."
   }

   # Get the fitness program
   program = state.fitness_plan.workout_program

   # Create the weekly schedule
   training_days = [
       "Monday: " + program[0],
       "Tuesday: Recovery day. Light activity or mobility work.",
       "Wednesday: " + program[1],
```

```
        "Thursday: Recovery day. Light activity or mobility work.",
        "Friday: " + program[2],
        "Saturday: Recovery or optional ROTC team training.",
        "Sunday: Rest and recovery."
    ]

    # Add Ranger Challenge preparation if applicable
    if state.profile.ranger_challenge:
```

3. Example calendar output

Here is an example of what the page would look like:

ROTC Training Calendar

University of Delaware · MS2

ROTC Development Focus

Continue developing basic military and leadership skills.

Weekly Training Schedule

Monday — Improvement Workout 1

Focus on the weakest AFT event.

Tuesday — Recovery

Light activity or mobility work.

Wednesday — Improvement Workout 2

Focus on the second weakest AFT event.

Thursday — Recovery

Light activity or mobility work.

Friday — Maintenance Workout

Maintain the two strongest AFT events.

Saturday — Ranger Challenge

Team practice, coordinated with the instructor.

Sunday — Rest

Recovery.

Illustrative schedule. Actual ROTC commitments and training should be coordinated with the battalion and fitness instructors.

4. How the calendar connects to the rest of your website

Your website now follows this sequence:

Profile

School, class year, Ranger Challenge

AFT Score

Five event scores

Fitness Plan

Two best events and two worst events

Calendar

Weekly schedule generated from the fitness plan and profile

One limitation to keep in mind: This version generates a repeating weekly schedule, not a date-specific calendar. It also uses the first three entries in your existing `workout_program` list, so keep those entries in the order established by your fitness plan route.

can you explain how to do this in drafter

Absolutely. Since you're building this in **Drafter using Python in Thonny**, I'll explain how to build the calendar using the Drafter components rather than HTML or a separate calendar library for displaying it.

The goal is to create a calendar that looks like this:

- November 2026 at the top.
- Sunday through Saturday across the top.
- Each date in its own box.
- Workout information inside the appropriate date.
- Buttons to move between months.

1. Understand how we will build the calendar

Drafter has a few components that will help us:

Drafter component	Purpose

`Page()`	Creates the page.
`Text()`	Displays text.
`Grid()`	Creates the calendar's columns and rows.
`Column()`	Organizes information vertically.
`Row()`	Organizes information horizontally.
`Button()`	Allows users to navigate months.

The important component is `Grid()`. We want seven columns because there are seven days in a week.

2. Update your Calendar dataclass

First, we need to keep track of the month and year.

```
@dataclass
class Calendar:
   training_days: list = field(default_factory=list)
   month: int = 11
   year: int = 2026
```

Why?

- `month` tells us which month to display.
- `year` tells us which year to display.
- `training_days` stores the workouts.

3. Import Python's calendar module

At the top of your main file, add:

```
import calendar as cal
```

This gives us access to a function called:

```
cal.monthcalendar(year, month)
```

For November 2026, it returns:

```
[
   [0, 0, 0, 0, 0, 0, 1],
   [2, 3, 4, 5, 6, 7, 8],
   [9, 10, 11, 12, 13, 14, 15],
   [16, 17, 18, 19, 20, 21, 22],
   [23, 24, 25, 26, 27, 28, 29],
   [30, 0, 0, 0, 0, 0, 0]
]
```

Notice that:

- Each inner list represents one week.
- Each number represents a date.
- `0` means that the date belongs to the previous or following month.

This is exactly what we need!

4. Build the calendar grid

Here is the basic structure I would use.

Important: Drafter's component names and arguments depend on the version you're using. The following is illustrative Python-style pseudocode, not guaranteed Drafter syntax. We should verify your installed component names before putting it into your project.

The structure we want is:

```
@route
def calendar(state: State) -> Page:

   year = state.calendar.year
   month = state.calendar.month

   month_dates = cal.monthcalendar(year, month)

   return Page(state, [
       "ROTC Training Calendar",
```

```
        month_name,

        Grid([
            # Sunday
            # Monday
            # Tuesday
            # Wednesday
            # Thursday
            # Friday
            # Saturday
        ]),

        Button("Previous Month", "previous_month"),
        Button("Next Month", "next_month")
    ])
```

The idea is that each week becomes a row, and each day becomes a cell.

5. How to build each individual date

For every date, we want a vertical arrangement:

```
Column([
    Text(str(day)),
    Text("Workout information")
])
```

For example, November 2 might display:

2

Deadlift workout

Strength training

You would repeat this for every date in the month.

6. Add the month navigation

We need two routes to change the month.

```
@route
def next_month(state: State) -> Page:

    state.calendar.month += 1
```

```
    if state.calendar.month > 12:
        state.calendar.month = 1
        state.calendar.year += 1

    return calendar(state)
```

And:

```
@route
def previous_month(state: State) -> Page:

    state.calendar.month -= 1

    if state.calendar.month < 1:
        state.calendar.month = 12
        state.calendar.year -= 1

    return calendar(state)
```

These routes update the month and return the user to the calendar.

7. Let's build it using the Drafter version you're learning

Before I write the complete working code, I want to make sure we use the correct components.

Which of these have you learned or used in Drafter?

Which Drafter components do you recognize?

Select all that apply.

Grid

Column

Row

Text

Button

Table

Other components you've used (optional)

I want to build this using the same Drafter conventions you've been learning, rather than introduce a completely different way of building websites.

The Drafter components I recognize are: Column, Row, Button. Other components: . Please build the calendar using these and explain each step.

Perfect. Since you've worked with `Column`, `Row`, and `Button`, we can build the calendar using just those components and Python lists. We won't need `Grid`, `Table`, or HTML.

The idea is to make **seven Columns (one for each day of the week), organized into Rows (one for each week).**

1. How the calendar will be organized

Think of the calendar as a collection of rows and columns:

November 2026

Sun	Mon	Tue	Wed	Thu	Fri	Sat
1						
2						
3						
4						
5						
6						
7						
8						
9						
10						
11						
12						
13						
14						
15						
16						
17						
18						
19						
20						

```
21
22
23
24
25
26
27
28
29
30
```

Conceptual layout of the calendar.

In Drafter, each week will be a `Row`, and each day will be a `Column`.

2. Update your Calendar dataclass

We need to store the month and year so the calendar can change when the user clicks the navigation buttons.

```
@dataclass
class Calendar:
    training_days: list = field(default_factory=list)
    month: int = 11
    year: int = 2026
```

You should also add this import at the top of your file:

```
import calendar as cal
```

3. Build the calendar route

Here is the code using the components you recognize.

```
@route
def calendar(state: State) -> Page:

    year = state.calendar.year
    month = state.calendar.month

    # Get the weeks of the selected month
    month_dates = cal.monthcalendar(year, month)

    # Create the weekday headings
    calendar_rows = [
```

```
        Row([
            Column(["Sunday"]),
            Column(["Monday"]),
            Column(["Tuesday"]),
            Column(["Wednesday"]),
            Column(["Thursday"]),
            Column(["Friday"]),
            Column(["Saturday"])
        ])
    ]

    # Create each week
    for week in month_dates:

        week_row = []

        for day in week:

            if day == 0:
                week_row.append(Column([" "]))
            else:
                week_row.append(
                    Column([
                        str(day)
                    ])
```

Let's break down the important parts

A. Creating the weekday headings

```
calendar_rows = [
    Row([
        Column(["Sunday"]),
        Column(["Monday"]),
        Column(["Tuesday"]),
        Column(["Wednesday"]),
        Column(["Thursday"]),
        Column(["Friday"]),
        Column(["Saturday"])
    ])
```

```
]
```

- `calendar_rows` is a list containing all the calendar rows.
- The first `Row` contains the seven weekdays.
- Each `Column` represents one day.

B. Creating each week

```
for week in month_dates:

   week_row = []

   for day in week:
```

The first loop goes through each week. The second loop goes through each day in that week.

C. Creating the individual date boxes

```
if day == 0:
   week_row.append(Column([" "]))
else:
   week_row.append(Column([str(day)]))
```

If the date is zero, we leave the box empty. Otherwise, we create a column containing that date.

D. Adding each week to the calendar

```
calendar_rows.append(Row(week_row))
```

This takes the seven date columns and puts them into one horizontal row.

Finally:

```
*calendar_rows,
```

The `*` unpacks the list so each row becomes an individual element in the `Page` list.

4. Add the month navigation routes

These allow the user to move between months.

```
@route
def next_month(state: State) -> Page:

   state.calendar.month += 1

   if state.calendar.month > 12:
       state.calendar.month = 1
       state.calendar.year += 1

   return calendar(state)



@route
def previous_month(state: State) -> Page:

   state.calendar.month -= 1

   if state.calendar.month < 1:
       state.calendar.month = 12
       state.calendar.year -= 1

   return calendar(state)
```

For example:

- November → Next → December
- December → Next → January of the following year
- January → Previous → December of the previous year

5. Add the workouts to the dates

Once the basic calendar works, we can add workout information inside each date's `Column`.

For example, instead of:

```
Column([
   str(day)
```

```
])
```

We can eventually use:

```
Column([
   str(day),
   "Deadlift Workout"
])
```

We can make the workout depend on the date and your `state.fitness_plan.workout_program`.

6. Test the calendar before adding workouts

I recommend doing this in stages:

1. Get November 2026 to display correctly.
2. Make sure Sunday through Saturday line up.
3. Test Previous Month and Next Month.
4. Add the workout information to each date.
5. Eventually allow users to click a date to see its full training details.

One Drafter-specific note: The code above follows the component structure we've discussed, but I haven't verified that your installed Drafter version accepts dynamically unpacked `Row` and `Column` objects or supports `Column` as a seven-column layout. If it produces an error, share the exact error and we can adapt it to your version rather than switching to HTML.

this is what drafter website says,

Table

Group: Data display

Description

A `Table` renders rows and columns. It works best with a list of dataclasses: the field names become the header row, and each instance becomes a row of the table. Since most projects keep exactly this kind of nested data in state, a `Table` can often display it with a single line of code.

Syntax

```
None
Table(rows)
Table(rows, header)
```

Parameters

Parameter	Type	Default	Meaning
`rows`	`list`	required	A list of dataclasses (header generated from fields), a list of lists (each inner list a row), or a single dataclass (rendered as field/value rows).
`header`	`list`	auto or none	Header cells, when you want different ones than the generated names.

Examples

```
None
from drafter import *
from dataclasses import dataclass


@dataclass
class Expedition:
    destination: str
    days: int
    survived: bool


@route
```

```
def index() -> Page:
    return Page([
        Header("Expedition Log"),
        Table([
            Expedition("the volcano", 3, True),
            Expedition("the couch", 1, True),
            Expedition("the DMV", 1, False)
        ]),
        Header("Same data, lists and a custom header", 2),
        Table([
            ["the volcano", 3],
            ["the couch", 1]
        ], ["Place", "Days"])
    ])


start_server()
```

Notes

- Rows usually come straight from state (`Table(state.expeditions)`), so the table reflects the list each time the page renders; the shows this pattern in a complete project.
- Cell values render the same way page content does: numbers do not need `str()`, and cells can contain components.
- In tests, match tables structurally: `assert_has(page, Table([Expedition("the couch", 1, True)]))`. Searching for the cell text instead finds nothing, because text matching does not reach inside rows.
- An empty list renders as an empty table; consider whether your page should show a message instead when there are no rows yet.

Perfect! This gives us a better way to build your calendar. Instead of using `Column()` and `Row()`, we can use Drafter's `Table()` **component** to create the calendar grid.

The important part is that `Table()` accepts a list of lists, where each inner list represents one row.

1. How we can structure the calendar

A calendar has seven columns, one for each day of the week:

```
None
Table([    ["Sunday", "Monday", "Tuesday", "Wednesday",
"Thursday", "Friday", "Saturday"],    [1, 2, 3, 4, 5, 6, 7],
[8, 9, 10, 11, 12, 13, 14],    [15, 16, 17, 18, 19, 20, 21],
[22, 23, 24, 25, 26, 27, 28],    [29, 30, "", "", "", "",
""]])
```

This is just an example of the layout. We will generate the dates automatically so it works for every month.

2. Building the calendar route

We can use Python's built-in `calendar` module to determine which day each month starts on and how many days it contains.

First, add this import at the top of your file:

```
None
import calendar as cal
```

Then your calendar route can look like this:

```
None
@routedef calendar(state: State) -> Page:    month =
state.calendar.month    year = state.calendar.year
month_dates = cal.monthcalendar(year, month)    calendar_rows
= [        ["Sunday", "Monday", "Tuesday", "Wednesday",
"Thursday", "Friday", "Saturday"]    ]    for week in
month_dates:        calendar_row = []        for day in week:
if day == 0:                calendar_row.append("")
```

```
else:                calendar_row.append(str(day))
calendar_rows.append(calendar_row)    return Page(state, [
"ROTC Training Calendar",        cal.month_name[month] + " " +
str(year),        Button("Previous Month", "previous_month"),
Button("Next Month", "next_month"),
Table(calendar_rows),        Button("Home", "index")    ])
```

3. Month navigation

We can keep the navigation routes simple:

```
None
@routedef next_month(state: State) -> Page:
state.calendar.month += 1    if state.calendar.month > 12:
state.calendar.month = 1        state.calendar.year += 1
return calendar(state)@routedef previous_month(state: State)
-> Page:    state.calendar.month -= 1    if
state.calendar.month < 1:        state.calendar.month = 12
state.calendar.year -= 1    return calendar(state)
```

4. What this accomplishes

- Displays the correct days for each month.
- Automatically adjusts for months with different lengths.
- Starts each week on Sunday.
- Allows users to navigate between months and years.

One limitation: This first version displays the calendar dates, but it does not yet display workouts inside the date cells.

I recommend getting this version running first. Then we can build the next part: storing training days in your it makes it for novemeber that the first starts on a saturday but it is actaully a sunday

You're right! The issue is most likely that Python's `calendar.monthcalendar()` starts weeks on **Monday by default**, while we want Sunday.

Add this line before generating `month_dates`:

```
cal.setfirstweekday(cal.SUNDAY)
```

So the beginning of your calendar route should look like this:

```
@route
def calendar(state: State) -> Page:
   month = state.calendar.month
   year = state.calendar.year

   cal.setfirstweekday(cal.SUNDAY)
   month_dates = cal.monthcalendar(year, month)
```

This tells Python to organize the calendar with Sunday as the first day of each week.

Why this fixes it: November 1, 2026, is a Sunday. With Sunday as the first weekday, the `1` will appear in the first column instead of the Saturday column.

Convert to PDF

how can we add data underneither the numbers

We can do this by changing what we put inside each calendar cell. Right now, each cell only contains the date number. We can make each cell contain the **date number and the workout information underneath it**.

Since Drafter's `Table()` accepts components inside its cells, we can use a `Text()` component or a string with a newline, depending on how you want the calendar to look.

1. Example of what we want

For example, a calendar cell could look like:

Sun
Mon
Tue
Wed

Thu

Fri

Sat

1

Run

30 min

2

3

Strength

Upper body

4

5

6

7

Illustration of the intended layout.

2. How to change your Python code

Currently, you probably have something like:

```
if day == 0:
   calendar_row.append("")
else:
   calendar_row.append(str(day))
```

We want to change the `else` section so it adds the date and its associated workout.

For example, if you store workouts in your `Calendar` dataclass as a list of strings, we could eventually use:

```
if day == 0:
   calendar_row.append("")
else:
   calendar_row.append(str(day) + "\nWorkout")
```

This is a simple demonstration. It would display the word Workout underneath every date.

3. One question before we build the actual version

How are you planning to store the workouts in your `Calendar` dataclass?

For example, would you prefer each training day to contain:

- A date (such as November 1, 2026)
- A workout name (such as Strength Training)
- A description (such as Upper-body workout)

I recommend storing those three pieces of information together so the calendar can automatically display the correct workout under the correct date.

Can you fix the slight errors in the code

I went through your entire ROTC Cadet Dashboard and corrected the errors while keeping your original organization, dataclasses, functions, and Drafter approach.

Corrected Python file

Ready to open in Thonny
Syntax checked

Download the corrected ROTC Cadet Dashboard

What I corrected

Issue	Correction

Duplicate `Calendar` dataclass	Removed the duplicate definition.
Calendar weekday alignment	Set Sunday as the first day and corrected the headings.
Empty fitness plan	Added a check so the calendar doesn't try to access workouts before a plan exists.
AFT time scoring	Added conversion for scoring-table times written as `M:SS`.
AFT time display	Added `seconds_to_time()` so saved times display correctly when reopening the AFT page.
Missing profile information	Added checks before calculating AFT scores.
Invalid time entries	Added basic time-format validation.

Important notes before running it

1. Keep `rotc_cadet_dashboard_fixed.py` in the same folder as `aft_scoring_data_table.py`.
2. Make sure you run the corrected file rather than your original file.
3. I checked that the Python file has valid syntax, but I haven't tested it against your separate AFT scoring tables or your installed Drafter version.

One feature is still a placeholder: The calendar alternates between the two weakest AFT events and displays a 30-minute workout. It does not yet save individually assigned workouts to specific dates. We can build that next.

rotc_cadet_dashboard_fixed.pyCode

Convert to PDF

Make sure you also have the docstring and can you from bakery import assert_equal and make a lot of assert_equal tests

I've updated the complete project to include the **module docstring**, `from bakery import assert_equal`, and **79 test cases**.

ROTC Cadet Dashboard – Updated

Updated

Download the updated Python file

What's included

- Full project docstring at the top.
- `from bakery import assert_equal`.
- 79 `assert_equal` tests.
- Original Drafter pages and navigation.
- AFT scoring and time conversion tests.
- Age-group boundary tests.
- Calendar date alignment tests.
- Dataclass default-value tests.

What the tests cover

Test category	What is tested
Time conversion	Converting M:SS into seconds and back
Invalid times	Rejecting incorrectly formatted times
Age groups	Testing age ranges and boundaries
AFT scoring	Higher-is-better and lower-is-better scoring
Profile	Gender formatting and defaults

Fitness plan	Independent lists for different plans
Calendar	November 2026 starts on Sunday
State	Correct default values

Important

I verified that the Python file passes a syntax check. However, the `bakery` package isn't installed in my execution environment, so I couldn't execute the 79 tests here.

When you run it in Thonny, make sure `bakery` and your `aft_scoring_data_table.py` file are available. The tests will execute before `start_server(State())` starts the website.

rotc_cadet_dashboard_fixed.pyCode

There are no doc string. After every function there should be '''

You're correct. You want a **docstring immediately after every function definition**, using three single quotation marks (`'''`), not just one general docstring at the top of the project.

I've updated the file so every function has its own description, including:

- Profile and navigation functions
- AFT scoring functions
- Fitness plan functions
- Calendar functions
- Helper and testing functions

Syntax checked

Download the corrected Python file with docstrings

I also kept the `assert_equal` tests and verified that every function has a docstring.

rotc_cadet_dashboard_with_docstrings.pyCode

Convert to PDF

For doctsring there should be 3 lines, The first line is what the goal of the function is, the second line is the arguments/peramiters, the 3rd line is what will be returned

Got it! Each function now has a **three-line docstring** with the exact structure you're describing:

1. **Goal:** What the function is supposed to accomplish.
2. **Parameters:** What information the function receives.
3. **Returns:** What the function gives back.

For example:

```
def get_age_group(age: int) -> str:
    '''Goal: Find the AFT scoring age group for a cadet's age.
    Parameters: age (int), the cadet's age in years.
    Returns: The matching age-group name; raises ValueError if under 17.'''
```

21 functions updated

Download the updated ROTC Dashboard with three-line docstrings

I also verified that all 21 functions have the three required lines and that the file passes Python syntax validation.

rotc_cadet_dashboard_three_line_docstrings.pyCode

Convert to PDF

Can you add into the program to make it so every monday of the calendar says- Morning PT - Short Intervals, Wednsday PT- 4 mile Ruck, and then Thursday PT- Long distance Intervals. Also add in rest days after the end of the cyle. Can you make sure there is doc string as well as making the overall User interface is a lot cleaner and much easier to use.

I can make those changes. However, data analysis is temporarily unavailable, so I couldn't finish creating the updated Python file. It may become available again in approximately 22 hours and 49 minutes; that estimate isn't guaranteed.

I have identified the changes needed for your current code:

1. Weekly PT schedule

Day	Calendar activity
Monday	Morning PT – Short Intervals
Tuesday	Recovery / No scheduled PT
Wednesday	PT – 4 mile Ruck
Thursday	PT – Long Distance Intervals
Friday	Rest Day
Saturday	Rest Day
Sunday	Rest Day

The schedule will repeat automatically every week, regardless of the month or year.

2. Cleaner user interface

I plan to improve the dashboard with:

- A clear home-page heading and navigation.
- Separate sections for profile information and AFT results.

- AFT scores displayed in a table.
- Fitness recommendations organized by strongest and weakest events.
- A calendar with Sunday–Saturday columns.
- A weekly schedule displayed above the calendar.

3. Docstrings

Every function will have exactly three lines using your required format:

```
def get_calendar_activity(year: int, month: int, day: int) -> str:
    '''Goal: Determine the scheduled activity for a calendar date.
    Parameters: year, month, and day (int), identifying the date.
    Returns: The activity description for that date as a string.'''
```

4. Testing

I'll also preserve `from bakery import assert_equal` and include tests for the weekly schedule, rest days, calendar dates, and existing AFT functions.

The updated file has not been created yet. I don't want to give you a download link to an incomplete or unverified program.

Convert to PDF

Thank you please, but please create the updated file

You're welcome! I have the project file attached, and I understand the updates you want.

Unfortunately, the file-creation tool is still temporarily unavailable due to a usage limit. I don't want to give you a broken or incomplete download link and claim the file is finished when it isn't.

Once file creation is available again, I can update the project with:

- Monday: Morning PT – Short Intervals
- Wednesday: PT – 4 Mile Ruck

- Thursday: PT – Long Distance Intervals
- Friday–Sunday: Rest Days
- Tuesday: Recovery / No Scheduled PT
- Three-line docstrings for every function
- A cleaner, easier-to-navigate website layout
- Updated calendar and navigation
- Additional tests to catch errors

I haven't created the updated file yet. I appreciate your patience.

Can you write the program out but only using the text file and dont create a updated .py file

Absolutely. I read your uploaded text file, and I'll write the **complete updated program below**, rather than creating a `.py` file.

I'm keeping your original Drafter structure, AFT scoring-table imports, profile fields, scoring functions, and navigation. I'm also making these changes:

- Remove the duplicate `Calendar` dataclass.
- Add the weekly PT schedule.
- Make the calendar start on Sunday, with the correct weekday headings.
- Display the scheduled activity inside each calendar date.
- Add a weekly schedule section above the calendar.
- Prevent the Fitness Plan page from crashing before AFT scores have been entered.
- Add the three-line docstring format to every function.
- Improve page organization and navigation.

Weekly schedule used:

Day	Scheduled activity

Monday	Morning PT – Short Intervals
Tuesday	Recovery / No Scheduled PT
Wednesday	PT – 4 Mile Ruck
Thursday	PT – Long Distance Intervals
Friday	Rest Day
Saturday	Rest Day
Sunday	Rest Day

Complete updated program

Copy this into your main Drafter program, replacing the contents of your current main file.

```
"""
ROTC Cadet Dashboard

This program allows ROTC cadets to manage profile information,
calculate AFT scores, view fitness recommendations, and follow
a weekly training calendar.
"""

from drafter import *
from dataclasses import dataclass, field
from aft_scoring_data_table import *
import calendar as cal


# -------------------------
# CALENDAR SETTINGS
# -------------------------

# Make Sunday the first day displayed in the calendar.
cal.setfirstweekday(cal.SUNDAY)
```

```
# ------------------------
# DATACLASSES
# ------------------------

@dataclass
class Profile:
   age: int = 0
   gender: str = ""
   school: str = ""
   class_year: str = "MS1"
   ranger_challenge: bool = False
```

Important things to know before running it

1. **Keep** `aft_scoring_data_table.py` **in the same project folder.** Your program still imports the five AFT scoring tables from that file.
2. **The calendar now starts on Sunday.** November 1, 2026, should appear under Sunday.
3. **The weekly schedule repeats automatically.** It does not depend on the month or the date the calendar starts.
4. **Tuesday is currently set to recovery/no scheduled PT.** You can change that in `get_calendar_activity()` if Tuesday should have a different activity.
5. **The calendar's activity text depends on how Drafter displays newlines inside** `Table` **cells.** The separate weekly schedule table ensures the full schedule is visible even if the calendar cells do not display multiple lines as expected.

One important AFT scoring note

Your original code stores the AFT results in seconds for the Sprint-Drag-Carry, Plank, and Run events. I preserved that approach. The scoring tables in `aft_scoring_data_table.py` must also use seconds for those events.

I also preserved your existing AFT scoring-table logic rather than changing the scoring standards. Make sure the tables you imported match the scoring standards your project is intended to use.

My recommendation: Run this version in Thonny first. Test the Profile, AFT Score, Fitness Plan, and Calendar pages individually. If Drafter reports an error, send me the exact error message and the line number so we can fix the issue without unnecessarily changing the rest of your program.

3. Most important concerns for a CS1 assignment

I would prioritize these five areas:

1. Exception handling

Not taught in depth

`try`, `except`, and `raise ValueError()`

2. Lambda functions

Not taught

`lambda x: x[1]`

3. Sorting with custom keys

Not established

`sorted(event_scores, key=...)`

4. Dataclass default factories

Advanced application

`field(default_factory=list)`

5. Python calendar library

Specialized library

`import calendar as cal`

These are the **five most important coding techniques I would flag as potentially beyond a typical CS1 course** in the updated ROTC program.

I would distinguish between techniques that are completely new and techniques that use familiar Python concepts in a more advanced way.

Analysis of the five techniques

Coding technique	Example from program	Why it may be beyond CS1
Exception handling	`try`, `except`, `raise ValueError()`	Introduces exception handling and manually raising errors.
Lambda functions	`lambda x: x[1]`	Uses anonymous functions instead of defining a regular function with `def`.
Sorting with custom keys	`sorted(event_scores, key=...)`	Sorts data based on a selected element within each tuple.
Dataclass default factories	`field(default_factory=list)`	Uses a specialized dataclass feature to create separate mutable defaults for each object.
Python calendar library	`import calendar as cal`	Introduces a specialized standard-library module for date and calendar calculations.

Where I would focus in your program

1. Exception handling

```
None

try:    state.aft_score.run = time_to_seconds(run)except ValueError as error:    return Page(state, [        "UNABLE TO CALCULATE SCORE",        str(error)    ])
```

Why it stands out: This introduces error handling, which allows the program to respond when an operation fails instead of crashing.

2. Lambda functions

```
None

sorted_scores = sorted(    event_scores,    key=lambda x: x[1])
```

Why it stands out: A lambda function creates a small, unnamed function inside another function call.

3. Sorting with custom keys

```
None

sorted_scores = sorted(    event_scores,    key=lambda x: x[1])
```

Why it stands out: The program sorts the event scores using the second item in each tuple rather than sorting the entire tuple alphabetically.

4. Dataclass default factories

```
None

@dataclassclass Fitness_plan:    best_events: list = field(default_factory=list)
```

Why it stands out: `default_factory` creates a new list whenever a new `Fitness_plan` object is created. This prevents different objects from unintentionally sharing the same list.

5. Python calendar library

```
None

import calendar as
calcal.setfirstweekday(cal.SUNDAY)month_dates =
cal.monthcalendar(year, month)
```

Why it stands out: This uses an imported library to calculate the layout of a calendar instead of manually calculating dates and weekdays.

My recommendation for your CS1 assignment

I would prioritize these techniques for review with your instructor:

1. **Lambda functions** – Most clearly introduces a new programming concept.
2. **Exception handling** – Introduces a new way of controlling program execution.
3. **Default factories** – Uses a more advanced feature of dataclasses.
4. **Custom sorting** – Combines sorting with functions and tuple indexing.
5. **Calendar library** – Uses an additional Python library.

One important distinction: `sorted()` itself is generally a CS1-level concept. The more advanced part is using a lambda function as its custom sorting key.

Also, if your instructor requires the project to use only concepts taught in class, you could replace the lambda sorting with a regular helper function and avoid some of these advanced techniques without losing the program's functionality.

Can you make sure this code fufiles all requriments that I will link -

Below are the requirements that your site must fulfill. Plan ahead so that meeting these requirements is easier as you develop your project.

- At least 7 routes.
- At least 5 Pages, or 10 FragmentsLinks to an external site., or a combination thereof.
- The implementation of the *State* dataclass must have at least 4 fields.
    - At least one field must be a list of class instances or a dictionary of class instances.
        - This means that there must be a second dataclass.

    - At least 4 state fields, including the list or dictionary of class instances, must each be meaningfully modified in at least one route; those 4 state fields cannot be constants.
- At least 3 input fields (TextBox, CheckBox, SelectBox, TextArea). They don't have to be the same type.
- At least 3 meaningful if statements.
- At least 1 loop that iterates over an attribute of the list of class instances or dictionary of class instances in a meaningful way.
- The site should have a legitimate purpose or functionality.
- No global variables (are fine).
- You must use the Drafter library.
- Any advanced coding techniques (i.e. that weren't covered in this course) used are thoroughly documented in the code.

Worked for 7s

Yes. I compared the ROTC Cadet Dashboard code we just wrote against all the requirements. **Your program meets most of the requirements, but there is one important requirement that it does not currently satisfy.**

The biggest issue is that your `State` dataclass needs a list of **class instances**, not simply a list of strings or tuples. We can fix this without changing the overall purpose of your website.

1. Requirement-by-requirement review

Meets requirement

= Already satisfied

Needs changes

= Needs an adjustment

Assignment requirement	Current program	Status
At least 7 routes	9 routes	Meets

At least 5 Pages or 10 Fragments	Multiple routes return Pages	Meets
State dataclass has 4+ fields	5 fields	Meets
List/dictionary of class instances	No list of class instances currently	Fix
4 State fields meaningfully modified	Profile, AFT scores, points, fitness plan, calendar	Meets
At least 3 input fields	Multiple TextBoxes and SelectBoxes	Meets
At least 3 meaningful if statements	Several examples	Meets
Loop over list of class instances	No such loop currently	Fix
Legitimate purpose	ROTC cadet dashboard	Meets
No global variables	No obvious user-created mutable globals; imported scoring tables should be constants	Likely meets
Must use Drafter	`from drafter import *`	Meets
Document advanced coding techniques	Some documentation exists, but needs expansion	Improve

2. Fix the biggest requirement: List of class instances

Currently, your `Calendar` dataclass looks like this:

```
None
@dataclassclass Calendar:    training_days: list =
field(default_factory=list)    month: int = 11    year: int =
2026
```

The problem is that the list does not contain instances of another dataclass.

We can create a `TrainingDay` dataclass.

Add this above your Calendar dataclass:

```
None
@dataclassclass TrainingDay:    day: str    activity: str
```

Then update your Calendar dataclass:

```
None
@dataclassclass Calendar:    training_days: list =
field(default_factory=list)    month: int = 11    year: int =
2026
```

Now `training_days` can contain instances of `TrainingDay`.

3. Populate the list and loop through it

Add this function to your program:

```
None
def create_training_schedule(state: State) -> None:
'''Goal: Create the weekly ROTC training schedule.
Parameters: state (State), containing the calendar
information.    Returns: None; modifies the training_days list
in the calendar.'''    state.calendar.training_days = [
```

```
TrainingDay("Monday", "Morning PT - Short Intervals"),
TrainingDay("Tuesday", "Recovery / No Scheduled PT"),
TrainingDay("Wednesday", "PT - 4 Mile Ruck"),
TrainingDay("Thursday", "PT - Long Distance Intervals"),
TrainingDay("Friday", "Rest Day"),
TrainingDay("Saturday", "Rest Day"),
TrainingDay("Sunday", "Rest Day")    ]
```

Next, inside your `calendar` route, replace the current `weekly_schedule_rows` section with:

```
None
    if len(state.calendar.training_days) == 0:
create_training_schedule(state)    weekly_schedule_rows = [
["Day", "Scheduled Activity"]    ]    for training_day in
state.calendar.training_days:
weekly_schedule_rows.append([            training_day.day,
training_day.activity        ])
```

This is particularly important because it satisfies two requirements:

1. Your State contains a list of instances of another dataclass.
2. Your program loops through that list and uses the information stored in each instance.

The loop is meaningful because it generates the weekly schedule displayed on your website.

4. Document the advanced coding techniques

Your assignment specifically says advanced techniques must be thoroughly documented.

I would add explanations like these directly above the relevant code.

Technique	Documentation to include
`field(default_factory=list)`	Creates a separate empty list for each dataclass instance, preventing different objects from sharing the same list.
`try` and `except`	Allows the program to handle invalid input without immediately terminating execution.
`raise ValueError()`	Manually raises an error when the supplied data does not meet the function's requirements.
`lambda`	Creates a short anonymous function used to determine which part of each tuple should be used for sorting.
`sorted(key=...)`	Sorts the event results using a selected value rather than the entire tuple.
`import calendar as cal`	Imports Python's calendar module and gives it the shorter name `cal`.

For example, document your sorting code like this:

```
None
# ADVANCED TECHNIQUE:# A lambda function creates a temporary function without# requiring a separate def statement.# The x[1] selects the second value in each event tuple.# sorted() uses that value to organize the event scores.sorted_scores = sorted(    event_scores,    key=lambda x: x[1])
```

This demonstrates that you understand the technique rather than simply using it.

5. Final recommendation

After making these changes, your project should satisfy the structural requirements.

Final project checklist

- At least 7 routes
- At least 5 Pages
- State has at least 4 fields
- At least 4 meaningful State fields
- At least 3 input fields
- At least 3 meaningful if statements
- Legitimate website purpose
- Drafter library
- List of class instances
- Loop through the list of class instances
- Thorough documentation of advanced techniques

The three unchecked items are addressed by the changes above. The checkmarks are based on reviewing the code, not on a successful run or instructor approval.

One additional recommendation: Since this is a CS1 assignment, I would consider replacing the lambda function with a regular helper function if lambda functions were not taught in your course. That would make your code easier to explain while preserving the sorting functionality.

can you create what the helper function would be that replaces lamda

Absolutely! You can replace the lambda function with a regular helper function using `def`.

1. Create this helper function

Place it above your `fitness_plan` route:

```
def get_event_points(event):
    '''Goal: Get the points earned for an AFT event.
    Parameters: event (tuple), containing an event name and point value.
    Returns: The point value stored in the second position.'''

    return event[1]
```

2. Replace your sorting code

Original code using lambda:

```
sorted_scores = sorted(
```

```
    event_scores,
    key=lambda x: x[1]
)
```

Updated code using your helper function:

```
sorted_scores = sorted(
    event_scores,
    key=get_event_points
)
```

3. How it works

Suppose your `event_scores` list looks like this:

```
event_scores = [
    ("Deadlift", 85),
    ("HRP", 90),
    ("Plank", 75),
    ("2 Mile Run", 80)
]
```

The helper function receives each tuple and returns its second value.

Event	Points
Deadlift	85
HRP	90
Plank	75
2 Mile Run	80

The `sorted()` function uses those returned point values to sort the events from lowest to highest.

Why this is better for your CS1 assignment: You are using a regular function that you can explain with concepts such as parameters, return values, tuples, and function calls instead of introducing lambda functions.

You do not lose any functionality, and the rest of your fitness-plan code can stay exactly the same.